

**TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
ĐẠI HỌC QUỐC GIA HÀ NỘI**



**BÁO CÁO MÔN HỌC
MACHINE LEARNING**

CHỦ ĐỀ:

DỰ ĐOÁN ĐIỂM THẺ TÍN DỤNG

GV hướng dẫn: Cao Văn Chung

Sinh viên thực hiện:

Họ Và Tên	MSV
Phan Nguyễn Mạnh Cường	23001843
Hồ Trọng Hiếu	23001874
Nguyễn Tiến Đạt	23001863

Hà Nội, năm 2025

Mục lục

1	Giới thiệu chung	5
1.1	Bối cảnh và vấn đề	5
1.2	Hạn chế của phương pháp đánh giá tín dụng truyền thống	5
1.3	Lợi ích của việc ứng dụng Machine Learning	5
1.4	Mục tiêu đặt ra	5
2	Cơ sở lý thuyết	6
2.1	Phân tích thành phần chính (PCA)	6
2.1.1	Định nghĩa	6
2.1.2	Ý tưởng	6
2.1.3	Các bước thực hiện PCA	7
2.2	Phân tích phân biệt tuyến tính (LDA)	8
2.2.1	Định nghĩa	8
2.2.2	Ý tưởng cho bài toán phân loại nhị phân	9
2.2.3	Xây dựng hàm mục tiêu	9
2.3	Giới thiệu phương pháp K-Nearest Neighbors	10
2.3.1	Định nghĩa	10
2.3.2	Nguyên lý hoạt động	10
2.3.3	Các độ đo khoảng cách	11
2.3.4	Chọn tham số K	11
2.3.5	Kỹ thuật tăng tốc	12
2.3.6	Ưu và nhược điểm	12
2.4	Softmax Regression	12
2.4.1	Định nghĩa và Bối cảnh	12
2.4.2	Các Hướng Tiếp cận từ Phân loại Nhị phân sang Đa lớp	12
2.4.3	Mô hình Toán học và Hàm Softmax	13
2.4.4	Hàm mất mát (Loss Function)	14
2.4.5	Cập nhật tham số và Hiệu chỉnh	14
2.5	Crammer-Singer SVM	15
2.5.1	Định nghĩa	15
2.5.2	Hàm mục tiêu	15
2.5.3	Các ràng buộc	15
2.5.4	Hàm mất mát hinge đa lớp	15
2.5.5	Quy tắc dự đoán	16
2.5.6	Ưu điểm của mô hình Crammer-Singer	16
2.6	K-means	16
2.6.1	Định nghĩa	16
2.6.2	Nguyên lý hoạt động	16
2.7	Phương pháp phân cụm Gaussian Mixture Model (GMM)	17

2.7.1	Mô hình toán học	17
2.7.2	Thuật toán Expectation-Maximization (EM)	17
2.8	Các chỉ số đánh giá	18
3	Tổng quan về bộ dữ liệu	18
3.1	Đánh giá dữ liệu ban đầu	18
3.2	Tiền xử lý dữ liệu	19
3.2.1	Đối với dữ liệu số ta thực hiện các thao tác sau:	19
3.2.2	Đối với dữ liệu category ta thực hiện các thao tác sau:	19
3.3	Trực quan hóa dữ liệu	19
4	Giảm chiều dữ liệu (PCA & LDA)	21
4.1	Phân tích thành phần chính (PCA)	21
4.2	Phân tích biệt thức tuyến tính (LDA)	23
5	Phân cụm dữ liệu (Thuật toán K-Means)	24
5.1	Cách chọn số cụm K	24
5.2	Xác định số cụm tối ưu	25
5.3	Phân cụm với K=3	26
5.3.1	Kết Luận	28
5.4	Phân cụm dữ liệu - GMM	29
5.4.1	Thực hiện phân cụm GMM	29
5.4.2	Kết quả phân cụm	29
5.4.3	Kết luận	30
6	Đánh giá mô hình K-Nearest Neighbors	30
6.1	Phương pháp đánh giá	30
6.1.1	Phân chia dữ liệu	30
6.1.2	Lựa chọn tham số K	30
6.2	Kết quả thực nghiệm	31
6.2.1	Tỷ lệ train:test = 80:20	31
6.2.2	Tỷ lệ train:test = 70:30	32
6.2.3	Tỷ lệ train:test = 60:40	32
6.2.4	Tỷ lệ train:test = 90:10	33
6.3	Kết luận về mô hình K-NN	34
6.3.1	Đánh giá về Accuracy	34
6.3.2	Đánh giá theo nhóm dữ liệu	34
6.3.3	Đánh giá theo tỷ lệ train:test	34
6.3.4	Đánh giá về tham số K	35
6.3.5	Cấu hình tốt nhất	35
6.3.6	Hạn chế	35
6.3.7	Độ phù hợp của K-NN	35

7	Đánh giá mô hình Softmax Regression	35
7.1	PHƯƠNG PHÁP ĐÁNH GIÁ	35
7.1.1	Phân chia dữ liệu	35
7.1.2	Lựa chọn tham số và chuẩn hóa dữ liệu	36
7.2	THỰC NGHIỆM	36
7.2.1	Quy trình huấn luyện	36
7.2.2	Kết quả thực nghiệm	36
7.3	KẾT LUẬN	38
7.3.1	Đánh giá về độ chính xác (Accuracy)	38
7.3.2	Đánh giá theo nhóm dữ liệu	39
7.3.3	Đánh giá theo tỉ lệ train:test	39
7.3.4	Phân tích ma trận nhầm lẫn và phân bố dự đoán	39
7.3.5	Nhận xét tổng quát	40
7.3.6	Kết luận	40
8	Đánh giá mô hình Crammer-Singer SVM	40
8.1	Kết quả thực nghiệm	40
8.1.1	Tỉ lệ train:test = 80:20	40
8.1.2	Tỉ lệ train:test = 70:30	41
8.1.3	Tỉ lệ train:test = 60:40	41
9	So sánh các mô hình	42
10	Mở rộng sang bài toán hồi quy	43
10.1	Mô hình SVR	43
10.1.1	Ý tưởng	43
10.1.2	Hàm mục tiêu	43
10.1.3	Kết quả thực nghiệm	44
10.1.4	Phân tích kết quả	45
10.1.5	Đánh giá tổng quan	46
10.2	Mô hình hồi quy dựa trên Softmax	46
10.2.1	Ý tưởng	46
10.2.2	Hàm mục tiêu	46
10.2.3	Kết quả thực nghiệm	47
10.2.4	Phân tích kết quả	48
10.2.5	Đánh giá tổng quan	49
	Tài liệu tham khảo	50

Danh sách hình vẽ

1	Hệ cơ sở trục chuẩn mới	7
2	Các bước giảm chiều PCA	8
3	So sánh chiều dữ liệu lên d1 và d2	8
4	Khoảng cách giữa các kỳ vọng và tổng các phương sai	9
5	Minh họa thuật toán K-Nearest Neighbors	11
6	Ví dụ về đầu vào và đầu ra của hàm softmax.	13
7	Thông số thống kê của một số thuộc tính trong bộ dữ liệu	18
8	Chỉ số thống kê sau khi xử lý dữ liệu	19
9	Biểu đồ phân bố cho các thuộc tính số	20
10	Biểu đồ phân bố các thuộc tính categorical	21
11	Biểu đồ Screeplot cho 22 PC	22
12	Biểu đồ tương quan giữa thành phần chính và mục tiêu	23
13	Biểu đồ phân tán và biểu đồ phân bố 2 thành phần của LDA	24
14	Đồ thị đánh giá mức tối ưu số cụm k	25
15	Trực quan hóa kết quả phân cụm với K=3	26
16	Ma trận nhầm lẫn	27
17	Iteration 0	27
18	Iteration 3 - 10	28
19	Iteration 20	28
20	Phân cụm GMM	29
21	Phân bố thực tế và dự đoán	31
22	Phân bố thực tế và dự đoán	32
23	Phân bố thực tế và dự đoán	33
24	Phân bố thực tế và dự đoán	34
25	Kết quả tỉ lệ 8:2	36
26	Kết quả tỉ lệ 7:3	37
27	Kết quả tỉ lệ 6:4	38
28	Kết quả tỉ lệ 8:2	40
29	Kết quả tỉ lệ 7:3	41
30	Kết quả tỉ lệ 6:4	42
31	Kết quả tỉ lệ 6:4	44
32	Kết quả tỉ lệ 6:4	45
33	Kết quả tỉ lệ 6:4	45
34	Kết quả đánh giá mô hình với các tỷ lệ	48
35	So sánh kết quả	49

1 Giới thiệu chung

1.1 Bối cảnh và vấn đề

Trong bối cảnh ngành tài chính ngày càng số hóa, việc đánh giá và quản lý rủi ro tín dụng đóng vai trò đặc biệt quan trọng đối với các tổ chức cho vay. Các ngân hàng và công ty tài chính phải liên tục ra quyết định về việc cấp tín dụng, hạn mức vay và lãi suất dựa trên khả năng hoàn trả của khách hàng. Trước đây, quá trình này chủ yếu dựa trên đánh giá thủ công hoặc các quy tắc định tính, dẫn đến việc tốn thời gian và tiềm ẩn rủi ro sai sót do yếu tố chủ quan. Nhằm nâng cao hiệu quả và tính chính xác trong việc phân loại khách hàng theo mức độ tín nhiệm, áp dụng hệ thống phân tích điểm tín dụng (Credit Score) dựa trên mô hình học máy (Machine Learning) là một bước tiến quan trọng.

1.2 Hạn chế của phương pháp đánh giá tín dụng truyền thống

Trong các hệ thống tài chính truyền thống, việc đánh giá điểm tín dụng của khách hàng chủ yếu dựa trên quy tắc định sẵn (rule-based) hoặc kinh nghiệm chuyên viên tín dụng. Phương pháp này có một số hạn chế rõ rệt:

1. **Phụ thuộc vào yếu tố con người:** Quá trình ra quyết định chịu ảnh hưởng mạnh bởi kinh nghiệm, cảm tính hoặc sai sót chủ quan của nhân viên thẩm định. Điều này có thể dẫn đến sự thiếu nhất quán giữa các quyết định và làm giảm độ tin cậy của hệ thống.
2. **Thiếu khả năng xử lý dữ liệu lớn và phức tạp:**
Khi lượng khách hàng tăng lên hàng chục hoặc hàng trăm nghìn, việc đánh giá thủ công trở nên không khả thi. Các quy tắc định tính không thể bao quát hết các mối quan hệ phi tuyến, đa chiều giữa các đặc trưng tín dụng như thu nhập, số lượng khoản vay, hành vi thanh toán hay lịch sử nợ.
3. **Thiếu khả năng dự báo và tối ưu hóa:** Phương pháp thủ công chủ yếu đánh giá theo hiện trạng thay vì dự báo rủi ro trong tương lai. Điều này khiến tổ chức tài chính khó phân tầng khách hàng hoặc phát hiện sớm nguy cơ nợ xấu.

1.3 Lợi ích của việc ứng dụng Machine Learning

Trước những hạn chế đó, việc ứng dụng mô hình học máy (Machine Learning) trong đánh giá tín dụng mang lại nhiều lợi ích vượt trội:

1. **Tự động hóa quy trình ra quyết định, giảm thiểu yếu tố cảm tính và sai sót con người.**
2. **Khai thác triệt để thông tin trong dữ liệu lớn, phát hiện các mối quan hệ phức tạp mà phương pháp truyền thống bỏ sót.**
3. **Cải thiện độ chính xác và tốc độ xử lý, giúp hệ thống phản hồi nhanh trong các tình huống thực tế.**

Nhờ vậy, mô hình học máy trở thành công cụ mạnh mẽ giúp doanh nghiệp đánh giá rủi ro tín dụng chính xác hơn, giảm thiểu nợ xấu, đồng thời tối ưu hóa quy trình phê duyệt tín dụng.

1.4 Mục tiêu đặt ra

Mục tiêu chính:

Mục tiêu chính của đề tài là xây dựng mô hình học máy tự động phân loại khách hàng vào các nhóm điểm tín dụng. Qua đó, hỗ trợ các tổ chức tài chính trong việc đánh giá rủi ro tín dụng và ra quyết định cho vay một cách chính xác, hiệu quả và tự động hơn.

Đề tài sẽ tập trung vào xử lý các đầu việc sau:

1. Tìm hiểu và tiền xử lý dữ liệu:

- Khám phá và hiểu rõ cấu trúc của tập dữ liệu điểm tín dụng.
- Tiến hành tiền xử lý dữ liệu: xử lý giá trị khuyết, dữ liệu ngoại lai, mã hóa các thuộc tính phân loại, và chuẩn hóa dữ liệu số.

2. Xây dựng và huấn luyện mô hình học máy:

- Lựa chọn và triển khai các thuật toán phân loại phổ biến Logistic Regression, K-Nearest Neighbors.
- Thực hiện tinh chỉnh siêu tham số (hyperparameter tuning) để tối ưu hiệu suất mô hình.

3. Đánh giá và so sánh mô hình:

- Sử dụng các chỉ số đánh giá như Accuracy, Precision, Recall, F1-score và ma trận nhầm lẫn để đo lường hiệu quả phân loại.
- So sánh các mô hình và lựa chọn mô hình có độ chính xác và khả năng tổng quát tốt nhất.

2 Cơ sở lý thuyết

2.1 Phân tích thành phần chính (PCA)

2.1.1 Định nghĩa

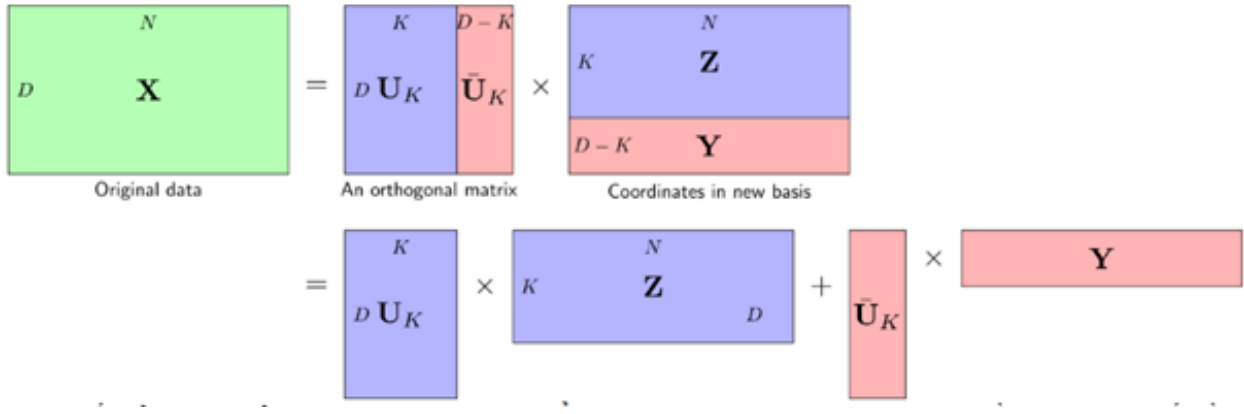
Là kỹ thuật giảm chiều dữ liệu được dùng phổ biến trong học máy và phân tích dữ liệu. Giúp giảm số thuộc tính của bộ dữ liệu sao cho dữ liệu được nhiều thông tin nhất. Hoạt động bằng cách thay đổi các thuộc tính ban đầu thành những thuộc tính mới không chồng chéo lên nhau và biểu diễn được nhiều thông tin nhất từ dữ liệu gốc.

Giúp loại bỏ thuộc tính dư thừa, tăng tốc độ tính toán, đơn giản hoá việc trực quan hoá qua đó giúp việc đánh giá, phân tích dữ liệu dễ dàng hơn.

2.1.2 Ý tưởng

Về mặt ý tưởng, thuật toán PCA tìm một hệ không gian mới và tối đa hóa phương sai dữ liệu của không gian mới đó. Sau đó lựa chọn ra n chiều có phương sai lớn nhất (giả thuyết rằng dữ liệu càng phân tán, phương sai càng lớn thì càng có giá trị)

Giả sử hệ cơ sở trực chuẩn mới là U và chúng ta muốn giữ lại K tọa độ trong hệ cơ sở mới này. Không mất tính tổng quát, giả sử đó là K thành phần đầu tiên.



Hình 1: Hệ cơ sở trực chuẩn mới

Quan sát hình vẽ trên với cơ sở mới $U = [U_K, \bar{U}_k]$ là một hệ trực chuẩn với U_k là ma trận con tạo bởi k cột đầu tiên của U . Với cơ sở mới này, ma trận dữ liệu có thể được viết thành:

$$X = U_k Z + \bar{U}_k Y \quad (1)$$

Mục đích của PCA là đi tìm ma trận trực giao U sao cho phần lớn thông tin được giữ lại ở phần màu xanh $U_k Z$ và phần màu đỏ $\bar{U}_k Y$ sẽ được lược bỏ và thay bằng một ma trận không phụ thuộc vào từng điểm dữ liệu.

2.1.3 Các bước thực hiện PCA

Quy trình thực hiện PCA sẽ như sau:

1. Tính vector kỳ vọng của toàn bộ dữ liệu:

$$\bar{x} = \frac{1}{N} \sum_{n=1}^N x_n \quad (2)$$

2. Trừ mỗi điểm dữ liệu đi vector kỳ vọng của toàn bộ dữ liệu:

$$\hat{x}_n = x_n - \bar{x} \quad (3)$$

3. Tính ma trận hiệp phương sai:

$$S = \frac{1}{N} \hat{X} \hat{X}^T \quad (4)$$

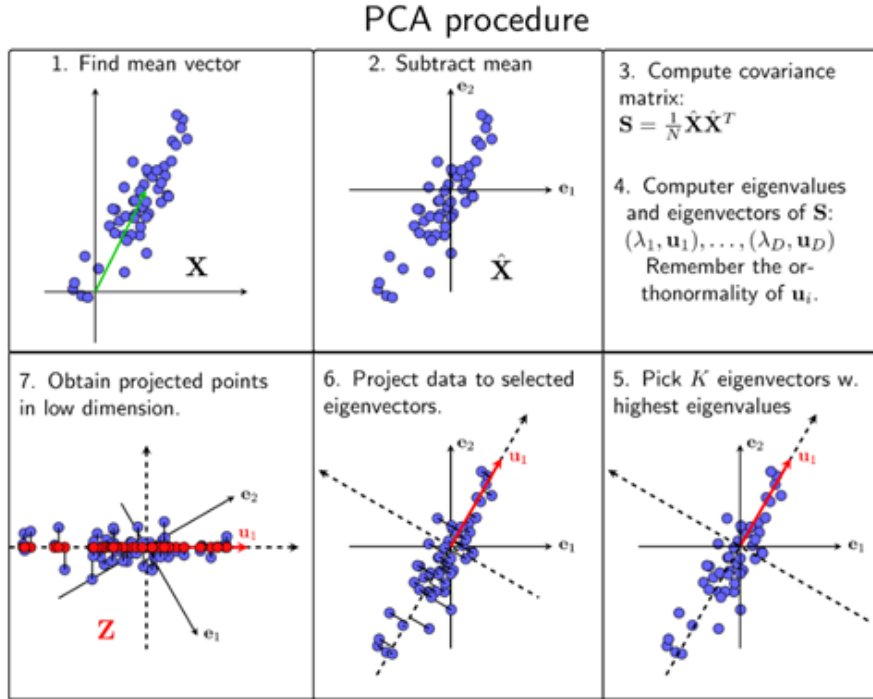
4. Tính các trị riêng và vector riêng có norm bằng 1 của ma trận này, sắp xếp chúng theo thứ tự giảm dần của trị riêng.

5. Chọn K vector riêng ứng với K trị riêng lớn nhất để xây dựng ma trận U_K có các cột tạo thành một hệ trực giao. K vectors này, còn được gọi là các thành phần chính, tạo thành một không gian con gần với phân bố của dữ liệu ban đầu đã chuẩn hoá.

6. Chiếu dữ liệu ban đầu đã chuẩn hoá $\hat{X}$ xuống không gian con tìm được

7. Dữ liệu mới chính là tọa độ của các điểm dữ liệu trên không gian mới.

$$x := U_k Z + \hat{x} \quad (5)$$

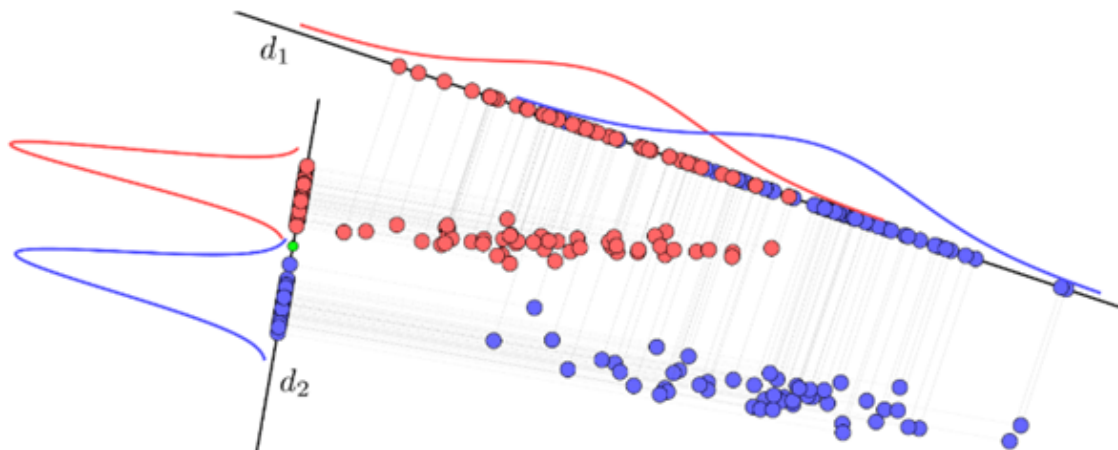


Hình 2: Các bước giảm chiều PCA

2.2 Phân tích phân biệt tuyến tính (LDA)

2.2.1 Định nghĩa

Trong nhiều trường hợp, ta không cần giữ lại lượng thông tin lớn nhất mà chỉ cần giữ lại thông tin cần thiết cho riêng bài toán. Xét ví dụ sau



Hình 3: So sánh chiếu dữ liệu lên d1 và d2

Xét hai đường thẳng d_1 và d_2 . Trong đó phương của d_1 gần với phương của thành phần chính nếu làm PCA, phương của d_2 gần với phương của thành phần phụ tìm được bằng PCA. Nếu ra làm giảm chiều dữ liệu bằng PCA, ta sẽ thu được dữ liệu gần với các điểm được chiếu lên d_1 . Lúc này việc phân tách hai class trở nên phức tạp vì các điểm đại diện cho hai

classes chồng lấn lên nhau. Ngược lại, nếu ta chiếu dữ liệu lên đường thẳng gần với thành phần phụ tìm được bởi PCA, tức d_2 , các điểm hình chiếu nằm hoàn toàn về hai phía khác nhau của điểm màu lục trên đường thẳng này. Với bài toán classification, việc chiếu dữ liệu lên d_2 vì vậy sẽ mang lại hiệu quả hơn. Việc phân loại một điểm dữ liệu mới sẽ được xác định nhanh chóng bằng cách so sánh hình chiếu của nó lên d_2 với điểm màu xanh lục này.

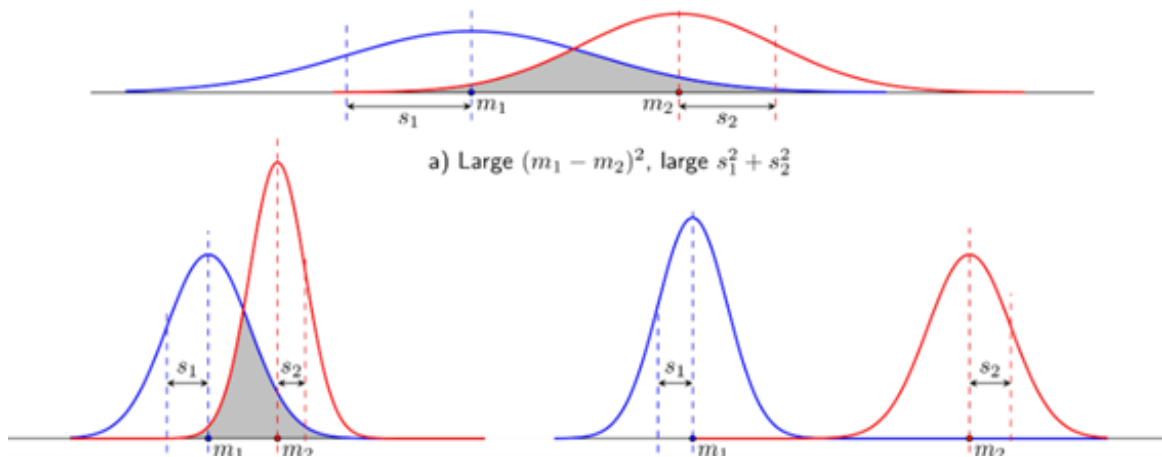
Qua ví dụ trên ta thấy, không phải việc giữ lại thông tin nhiều nhất sẽ luôn mang lại kết quả tốt nhất. Để khắc phục vấn đề này Linear Discriminant Analysis (LDA) được ra đời. Số chiều của dữ liệu mới sẽ nhỏ hơn hoặc bằng $C - 1$ trong đó C là số lớp phân loại. Từ ‘Discriminant’ được hiểu là những thông tin đặc trưng cho mỗi class, khiến nó không bị lẫn với các classes khác. Từ ‘Linear’ được dùng vì cách giảm chiều dữ liệu được thực hiện bởi một ma trận chiếu (projection matrix), là một phép biến đổi tuyến tính (linear transform).

2.2.2 Ý tưởng cho bài toán phân loại nhị phân

Quay lại với hình trên, các đường hình chuông thể hiện đồ thị của các hàm mật độ xác suất (probability density function - pdf) của dữ liệu được chiếu xuống theo từng class. Độ rộng của mỗi đường hình chuông thể hiện độ lệch chuẩn của dữ liệu. Dữ liệu càng tập trung thì độ lệch chuẩn càng nhỏ, càng phân tán thì độ lệch chuẩn càng cao. Khi được chiếu lên d_1 , dữ liệu của hai classes bị phân tán quá nhiều, khiến cho chúng bị trộn lẫn vào nhau. Khi được chiếu lên d_2 , mỗi classes đều có độ lệch chuẩn nhỏ, khiến cho dữ liệu trong từng class tập trung hơn, dẫn đến kết quả tốt hơn.

LDA hướng đến mục tiêu phân biệt rõ ràng nhất giữa các lớp dữ liệu, và để đạt được điều đó, cần thực hiện hai bước chính sau:

- Tối thiểu hai phương sai s_1^2, s_2^2 còn được gọi là các within-class variances.
- Tối đa bình phương khoảng cách giữa hai kỳ vọng $(m_1 - m_2)^2$ còn được gọi là between-class variance.



Hình 4: Khoảng cách giữa các kỳ vọng và tổng các phương sai

2.2.3 Xây dựng hàm mục tiêu

Như vậy, LDA là thuật toán đi tìm giá trị lớn nhất của hàm mục tiêu:

$$J(w) = \frac{(m_1 - m_2)^2}{s_1^2 + s_2^2} \quad (6)$$

Tiếp theo, chúng ta sẽ đi tìm biểu thức phụ thuộc giữa tử số và mẫu số trong vế phải của biểu thức vào w . Với S_b là một ma trận khoảng cách giữa các kỳ vọng (between-class variance), S_w là ma trận độ lệch chuẩn trong từng class (within-class

variance). Cả hai đều là ma trận đối xứng nửa xác định dương. Với tử số:

$$(m_1 - m_2)^2 = w^T (m_1 - m_2)(m_1 - m_2)^T w = w^T S_b w \quad (7)$$

Với mẫu số:

$$\begin{aligned} s_1^2 + s_2^2 &= \sum_{k=1}^2 \sum_{n \in C_k} (\mathbf{w}^T (\mathbf{x}_n - \mathbf{m}_k))^2 \\ &= \mathbf{w}^T \sum_{k=1}^2 \sum_{n \in C_k} (\mathbf{x}_n - \mathbf{m}_k)(\mathbf{x}_n - \mathbf{m}_k)^T \mathbf{w} \\ &= \mathbf{w}^T \mathbf{S}_w \mathbf{w} \end{aligned}$$

Như vậy bài toán LDA trở thành:

$$w = \arg \max_w \frac{w^T S_b w}{w^T S_w w} \quad (8)$$

Nghiệm w của bài toán LDA sẽ là nghiệm của phương trình đạo hàm hàm mục tiêu bằng 0. Sử dụng công thức $\nabla_w (wAw) = 2Aw$ nếu A là một ma trận đối xứng, ta có:

$$\begin{aligned} \nabla_{\mathbf{w}} J(\mathbf{w}) &= \frac{1}{(\mathbf{w}^T \mathbf{S}_w \mathbf{w})^2} (2\mathbf{S}_b \mathbf{w} (\mathbf{w}^T \mathbf{S}_w \mathbf{w}) - 2\mathbf{w}^T \mathbf{S}_b \mathbf{w}^T \mathbf{S}_w \mathbf{w}) = 0 \\ \iff \mathbf{S}_b \mathbf{w} &= \frac{\mathbf{w}^T \mathbf{S}_b \mathbf{w}}{\mathbf{w}^T \mathbf{S}_w \mathbf{w}} \mathbf{S}_w \mathbf{w} \\ \mathbf{S}_w^{-1} \mathbf{S}_b \mathbf{w} &= J(\mathbf{w}) \mathbf{w} \end{aligned}$$

Ở công thức trên ta giả sử rằng S_w là khả nghịch. Điều này không phải lúc nào cũng đúng nên có lúc ta sẽ cần xấp xỉ S_w bằng $\overline{S_w} \approx S_w + \alpha I$, với α là một số thực dương nhỏ. Vì S_w là nửa xác định dương nên ma trận mới có mọi giá trị riêng lớn hơn hoặc bằng giá trị riêng nhỏ nhất của $S_w + \alpha I$, tức là luôn lớn hơn $\alpha > 0$. Từ đó suy ra $\overline{S_w}$ là xác định dương vì có mọi giá trị riêng > 0 , nên nó khả nghịch.

2.3 Giới thiệu phương pháp K-Nearest Neighbors

2.3.1 Định nghĩa

K-nearest neighbor (KNN) là một trong những thuật toán học có giám sát đơn giản nhất trong Machine learning. Ý tưởng của KNN là dự đoán nhãn của điểm mới dựa trên k điểm gần nhất.

2.3.2 Nguyên lý hoạt động

Đầu vào:

- Tập huấn luyện (X, y)
- K – số láng giềng gần nhất
- Hàm khoảng cách (distance metric)

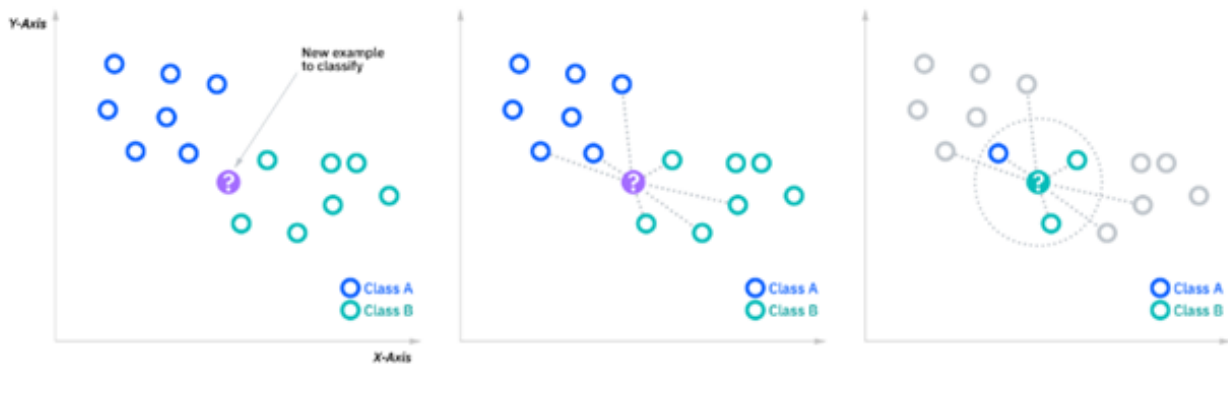
Các bước thực hiện:

Bước 1: Tính khoảng cách đối tượng cần phân lớp với tất cả các đối tượng trong tập huấn luyện

Bước 2: Sắp xếp khoảng cách theo thứ tự tăng dần

Bước 3: Lấy ra K điểm gần nhất – láng giềng

Bước 4: Xác định nhãn cho đối tượng dựa vào nhãn chiếm đa số



Hình 5: Minh họa thuật toán K-Nearest Neighbors

2.3.3 Các độ đo khoảng cách

a) Euclidean Distance

- Khoảng cách đường thẳng giữa 2 điểm
- Công thức: $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + \dots}$
- Được sử dụng phổ biến nhất

b) Manhattan Distance

- Khoảng cách theo đường đi dạng lưới
- Công thức: $d = |x_1 - x_2| + |y_1 - y_2| + \dots$
- Tốt khi dữ liệu có nhiều chiều

c) Cosine Distance

- Đo góc giữa 2 vector, không quan tâm độ lớn
- Thích hợp cho dữ liệu văn bản, thưa

2.3.4 Chọn tham số K

Nguyên tắc:

- K nhỏ (1-3): Nhạy cảm với nhiễu, dễ overfitting
- K lớn: Mượt nhưng có thể underfitting
- K lẻ: Tránh hòa phiếu trong phân loại nhị phân

Phương pháp chọn K:

- Dùng K-Fold Cross Validation:
 - Chia dữ liệu thành K phần
 - Thử nhiều giá trị k
 - Chọn k cho độ chính xác tốt nhất
- Thường $k = \sqrt{n}$ là điểm khởi đầu, n là số mẫu

2.3.5 Kỹ thuật tăng tốc

- KD-Tree (K-Dimensional Tree)
- Ball-Tree
- ANN (Approximate Nearest Neighbors)

2.3.6 Ưu và nhược điểm

Ưu điểm:

- Đơn giản, dễ hiểu; linh hoạt với ranh giới phi tuyến
- Không cần giả định phân phối; xử lý đa lớp/hồi quy tự nhiên

Nhược điểm:

- Tốn thời gian truy vấn khi dữ liệu lớn; nhạy với scale/nhiều
- Suy giảm trong không gian chiều cao; cần chọn k và độ đo phù hợp

2.4 Softmax Regression

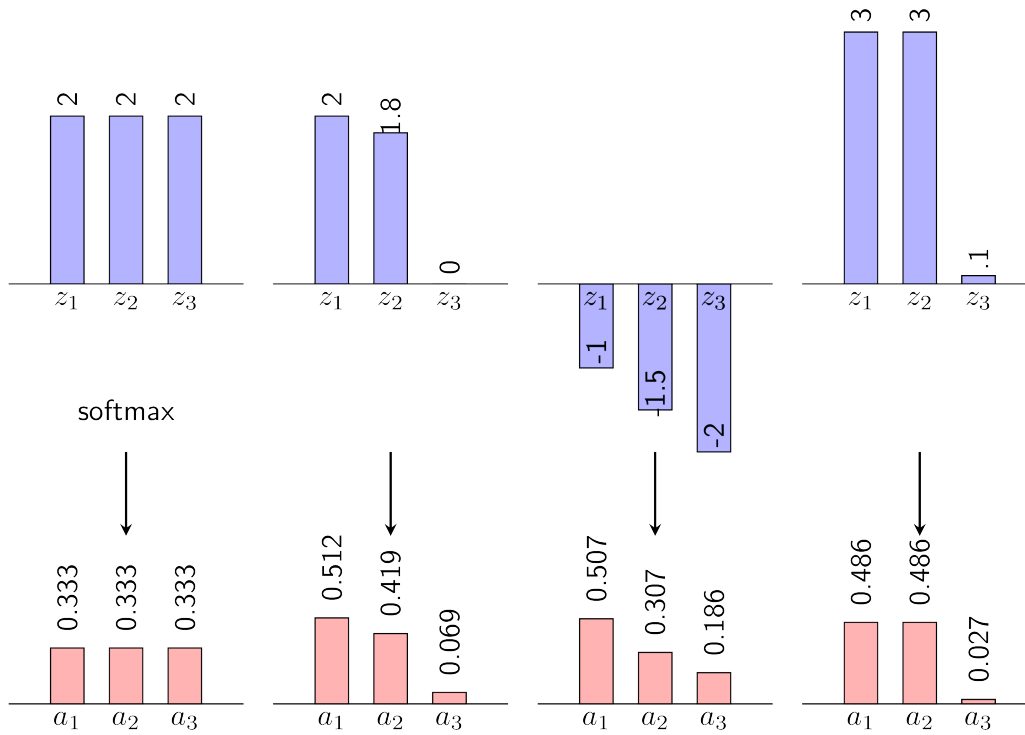
2.4.1 Định nghĩa và Bối cảnh

- **Softmax Regression** (hay Multinomial Logistic Regression, Mô hình Entropy cực đại) là phần mở rộng của Hồi quy Logistic, dùng cho bài toán **phân loại đa lớp** (*Multi-class classification*) với đầu ra $C > 2$ lớp ($C = \{1, 2, \dots, C\}$).
- **Binary Classifiers** (như Hồi quy Logistic nhị phân) chỉ xử lý 2 lớp ($y \in \{0, 1\}$ hoặc $y \in \{-1, 1\}$), và ranh giới phân lớp là một siêu phẳng ($H : \theta^T x = 0$).
- Mô hình này thuộc nhóm các **mô hình log-tuyến tính**, thường yêu cầu dữ liệu phải **hầu như tách được tuyến tính** (*nearly linearly separable*).

2.4.2 Các Hướng Tiếp cận từ Phân loại Nhị phân sang Đa lớp

Trước khi đến Softmax, các cách tiếp cận mở rộng B.C sang M.C bao gồm:

- **One-vs-One (OVO):** Xây dựng $\frac{C(C-1)}{2}$ bộ phân loại nhị phân, mỗi bộ phân loại một cặp lớp.
- **Hierarchical (Phân tầng):** Chia dữ liệu thành các nhóm con và phân loại theo tầng.
- **Binary Coding:** Mã hóa mỗi lớp bằng một vector nhị phân, cần $m = \lceil \log_2(C) \rceil$ bộ phân loại.



Hình 6: Ví dụ về đầu vào và đầu ra của hàm softmax.

- **One-vs-Rest (OVR):** Xây dựng C bộ phân loại, mỗi bộ phân biệt một lớp với *phần còn lại* (not class). Đây là cơ sở cho Softmax Regression, sử dụng *one-hot coding* cho các lớp.

Lưu ý quan trọng của OVR: Tổng xác suất $\sum_{i=1}^C P(y = i|x)$ có thể **khác 1** nếu sử dụng hàm Sigmoid độc lập cho từng lớp.

2.4.3 Mô hình Toán học và Hàm Softmax

Mô hình tuyến tính

Để áp dụng cho C lớp, mô hình Softmax cần C vector tham số $\theta_1, \dots, \theta_C$ (tham số tạo thành ma trận kích thước $(d+1) \times C$). Tổ hợp tuyến tính (log-odds) cho lớp k là:

$$z_k = \theta_k^T \mathbf{x}$$

Hàm Softmax

Để khắc phục nhược điểm của OVR (tổng xác suất khác 1), hàm **Softmax** được sử dụng để ánh xạ tổ hợp tuyến tính $\{z_1, \dots, z_C\}$ thành xác suất $P(y = k|x)$ sao cho $\sum_{c=1}^C P(y = c|x) = 1$:

$$P(y = k|x; \theta) = g(z_k) = \frac{\exp(z_k)}{\sum_{c=1}^C \exp(z_c)} = \frac{\exp(\theta_k^T \mathbf{x})}{\sum_{c=1}^C \exp(\theta_c^T \mathbf{x})}$$

Hàm Softmax có các tính chất: $g(z) \in (0, 1)$ và giữ nguyên thứ tự (nếu $z_i > z_j$ thì $g(z_i) > g(z_j)$).

Softmax-stable

Để khắc phục hiện tượng **tràn số** khi $\exp(z_i)$ quá lớn, phiên bản ổn định được sử dụng bằng cách trừ đi một hằng số Cnt (thường chọn $Cnt = \max_i z_i$):

$$P(y = k|\mathbf{x}) = \frac{\exp(z_k - Cnt)}{\sum_{c=1}^C \exp(z_c - Cnt)}$$

2.4.4 Hàm mất mát (Loss Function)

Hàm mất mát được xây dựng dựa trên **Ước lượng Hợp lý Cực đại (MLE)** hoặc **Cross-Entropy** (Entropy chéo).

Log-Likelihood Trung bình

Log-Likelihood trung bình của tập dữ liệu huấn luyện (dạng cần cực đại hóa) là:

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{n=1}^N \log P(y_n|\mathbf{x}_n; \boldsymbol{\theta}) = \frac{1}{N} \sum_{n=1}^N \left[(\boldsymbol{\theta}_{y_n}^T \mathbf{x}_n) - \log \left(\sum_{c=1}^C \exp(\boldsymbol{\theta}_c^T \mathbf{x}_n) \right) \right]$$

Hàm Tổn thất (Loss Function)

Hàm tổn thất $J(\boldsymbol{\theta})$ được định nghĩa là âm của Log-Likelihood (dạng cần **cực tiểu hóa**):

$$J(\boldsymbol{\theta}) = -\mathcal{L}(\boldsymbol{\theta})$$

2.4.5 Cập nhật tham số và Hiệu chỉnh

Đạo hàm hàm Tổn thất (Gradient)

Đạo hàm của Log-Likelihood $\mathcal{L}(\boldsymbol{\theta})$ theo tham số $\theta_{k,j}$ (tham số của lớp k , chiều j) là:

$$\frac{\partial}{\partial \theta_{k,j}} \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{n=1}^N [\delta(y_n = k) - P(y_n = k|\mathbf{x}_n; \boldsymbol{\theta})] (\mathbf{x}_n)_j$$

Trong đó:

- $\delta(y_n = k)$: Hàm chỉ thị (*indicator function*), bằng 1 nếu $y_n = k$, ngược lại bằng 0 (chính là thành phần one-hot coding $\bar{y}_{n,k}$).
- $[\dots]$: Là sai số $e_{k,n}$ giữa nhãn quan sát và xác suất dự đoán.

Cập nhật tham số (Gradient Descent)

Các tham số được cập nhật theo công thức:

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \cdot \frac{\partial J(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}}$$

với η là **learning rate**. Có hai phương pháp chính: **Stochastic Gradient Descent (SGD)** (cập nhật sau mỗi mẫu) và **Batch Gradient Descent (BGD)** (cập nhật sau toàn bộ dữ liệu).

Hiệu chỉnh L2 (Regularization)

Để tránh **overfitting**, mô hình thường thêm thành phần hiệu chỉnh L2 (Ridge) vào hàm tổn thất:

$$J(\boldsymbol{\theta})_{L2} = -\mathcal{L}(\boldsymbol{\theta}) + \frac{\lambda}{2} \sum_{c=1}^C \|\boldsymbol{\theta}_c\|^2$$

Đạo hàm hàm tổn thất có hiệu chỉnh L2 (cho toàn bộ N dữ liệu, dạng ma trận) là:

$$\frac{\partial J(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = \mathbf{X}(\mathbf{A} - \bar{\mathbf{Y}})^T + \lambda \boldsymbol{\theta}$$

Trong đó $\mathbf{A}$ là ma trận xác suất dự đoán và $\bar{\mathbf{Y}}$ là ma trận nhãn one-hot.

2.5 Crammer-Singer SVM

2.5.1 Định nghĩa

Crammer-Singer SVM là một phương pháp Support Vector Machine đa lớp trực tiếp, được đề xuất bởi Crammer và Singer, nhằm khắc phục hạn chế của các chiến lược phân rã như One-vs-Rest hay One-vs-One. Thay vì huấn luyện nhiều bộ phân loại nhị phân độc lập, Crammer-Singer tối ưu một hàm mục tiêu duy nhất cho tất cả các lớp, trong đó mỗi lớp được gán một vector trọng số riêng. Mô hình yêu cầu điểm dữ liệu phải có điểm số của lớp đúng lớn hơn lớp sai nguy hiểm nhất một khoảng margin xác định. Cách tiếp cận này tạo ra biên phân tách đa lớp toàn cục, nhất quán về hình học và giảm mâu thuẫn giữa các bộ phân loại nhị phân.

2.5.2 Hàm mục tiêu

Trong bài toán phân loại đa lớp với K lớp, mô hình Support Vector Machine (SVM) tìm tập các vectơ trọng số $\{w_k\}_{k=1}^K$ bằng cách tối thiểu hóa hàm mục tiêu:

$$\min_{\{w_k\}, \{\xi_n\}} \frac{1}{2} \sum_{k=1}^K \|w_k\|^2 + C \sum_{n=1}^N \xi_n \quad (9)$$

trong đó ξ_n là biến slack và $C > 0$ là tham số điều chỉnh mức độ phạt sai số.

2.5.3 Các ràng buộc

Bài toán tối ưu được giải dưới các ràng buộc:

$$\forall n, \forall k \neq y_n : w_{y_n}^\top x_n - w_k^\top x_n \geq 1 - \xi_n, \quad \xi_n \geq 0 \quad (10)$$

2.5.4 Hàm mất mát hinge đa lớp

Trong mô hình SVM đa lớp theo formulation của Crammer-Singer, hàm mất mát hinge đa lớp cho một mẫu huấn luyện (x_n, y_n) được xác định như sau:

$$\ell(x_n, y_n) = \max_{k \neq y_n} (1 + w_k^\top x_n - w_{y_n}^\top x_n) \quad (11)$$

Hàm mất mát này chỉ xét đến lớp sai có mức vi phạm biên lớn nhất, từ đó đảm bảo việc tối ưu margin được thực hiện một cách toàn cục.

2.5.5 Quy tắc dự đoán

Sau khi huấn luyện, nhãn của một mẫu mới x được dự đoán theo quy tắc:

$$\hat{y} = \arg \max_{k \in \{1, \dots, K\}} w_k^\top x \quad (12)$$

2.5.6 Ưu điểm của mô hình Crammer–Singer

Mô hình Crammer–Singer SVM có các ưu điểm sau:

- Không phân rã bài toán đa lớp thành các bài toán phân loại nhị phân riêng rẽ.
- Margin đa lớp được tối ưu toàn cục dựa trên lớp sai nguy hiểm nhất.
- Việc sử dụng biến slack chung giúp tạo ra biên quyết định nhất quán về mặt hình học.

Do đó, Crammer–Singer SVM có thể được xem là một dạng one-vs-rest đã được chuẩn hóa về mặt hình học, trong đó margin đa lớp được tối ưu toàn cục và chỉ lớp sai có mức vi phạm lớn nhất mới ảnh hưởng đến quá trình học.

2.6 K-means

2.6.1 Định nghĩa

Phân cụm (Clustering) là một phương pháp học không giám sát, trong đó dữ liệu chưa được gán nhãn được sử dụng để tìm ra các cấu trúc hoặc nhóm tiềm ẩn. Mục tiêu của phân cụm là chia tập dữ liệu thành các nhóm sao cho:

- Các điểm dữ liệu trong cùng một nhóm có mức độ tương đồng cao
- Các điểm dữ liệu thuộc các nhóm khác nhau có mức độ khác biệt lớn

K-means là một thuật toán phân cụm dựa trên tâm cụm, trong đó chúng ta tính toán khoảng cách giữa mỗi điểm dữ liệu và một tâm cụm để gán nó vào một cụm. Mục tiêu là xác định K nhóm trong tập dữ liệu.

2.6.2 Nguyên lý hoạt động

Bước 1: Lựa chọn số lượng cụm K

Xác định số cụm K mà chúng ta sẽ nhóm dữ liệu vào.

Bước 2: Khởi tạo tâm cụm ban đầu

Tâm cụm (Centroid) là trung tâm của một cụm dữ liệu, ban đầu chọn ngẫu nhiên các điểm dữ liệu và định nghĩa chúng là tâm cụm cho mỗi cụm.

Bước 3: Gán điểm dữ liệu vào cụm gần nhất

Ở bước này, chúng ta sẽ tính khoảng cách mỗi điểm dữ liệu và gán vào cụm có tâm gần nhất, thường sử dụng khoảng cách Euclidean.

Khoảng cách Euclidean:

$$d(x, \mu_k) = \sqrt{\sum_{j=1}^d (x_j - \mu_{kj})^2} \quad (13)$$

Trong đó:

- $x = (x_1, x_2, \dots, x_d)$: điểm dữ liệu

- $\mu_k = (\mu_{k1}, \mu_{k2}, \dots, \mu_{kd})$: tâm cụm thứ k
- d : số chiều dữ liệu

Bước 4: Cập nhật tâm cụm

Với mỗi cụm sau khi nhóm lại, ta cập nhật lại điểm trung tâm của chúng bằng cách lấy trung bình cộng.

Bước 5: Lặp lại cho đến khi hội tụ

Chúng ta sẽ tiếp tục lặp lại bước 3 và 4 cho đến khi có được các tâm cụm tối ưu và việc gán các điểm dữ liệu vào các cụm chính xác không còn thay đổi đáng kể nữa.

2.7 Phương pháp phân cụm Gaussian Mixture Model (GMM)

Gaussian Mixture Model (GMM) là một phương pháp phân cụm xác suất dựa trên giả định rằng dữ liệu được sinh ra từ một hỗn hợp của nhiều phân phối chuẩn (Gaussian) khác nhau [?]. Mỗi phân phối Gaussian đại diện cho một cụm tiềm ẩn trong dữ liệu.

2.7.1 Mô hình toán học

Hàm mật độ xác suất của GMM được định nghĩa như sau:

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \quad (14)$$

trong đó:

- K là số lượng thành phần Gaussian (số cụm)
- π_k là trọng số của thành phần thứ k , với $\sum_{k=1}^K \pi_k = 1$ và $\pi_k \geq 0$
- $\mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$ là hàm mật độ xác suất Gaussian với vector trung bình $\boldsymbol{\mu}_k$ và ma trận hiệp phương sai $\boldsymbol{\Sigma}_k$

Hàm mật độ Gaussian đa biến được biểu diễn:

$$\mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) = \frac{1}{(2\pi)^{d/2} |\boldsymbol{\Sigma}_k|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu}_k)^T \boldsymbol{\Sigma}_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k) \right) \quad (15)$$

với d là số chiều của dữ liệu.

2.7.2 Thuật toán Expectation-Maximization (EM)

Các tham số của GMM được ước lượng thông qua thuật toán EM, bao gồm hai bước lặp:

Bước E (Expectation): Tính xác suất hậu nghiệm (responsibility) của mỗi điểm dữ liệu thuộc về cụm k :

$$\gamma_{nk} = \frac{\pi_k \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)} \quad (16)$$

Bước M (Maximization): Cập nhật các tham số:

$$\mu_k^{\text{new}} = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} \mathbf{x}_n \quad (17)$$

$$\Sigma_k^{\text{new}} = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} (\mathbf{x}_n - \mu_k^{\text{new}})(\mathbf{x}_n - \mu_k^{\text{new}})^T \quad (18)$$

$$\pi_k^{\text{new}} = \frac{N_k}{N} \quad (19)$$

với $N_k = \sum_{n=1}^N \gamma_{nk}$ và N là tổng số điểm dữ liệu.

2.8 Các chỉ số đánh giá

- Accuracy: tỷ lệ dự đoán đúng trên tổng số dự đoán.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

- Confusion Matrix (Ma trận nhầm lẫn): bảng tóm tắt hiệu suất, cách các điểm dữ liệu được mô hình phân loại đúng và sai theo từng lớp.
- Precision, Recall, F1-Score (từ Classification Report)
Precision: tỉ lệ số điểm true positive trong số những điểm được phân loại là positive
Recall: tỉ lệ số điểm true positive trong số những điểm thực sự là positive
F1-Score: tỷ số dung hòa của Precision và Recall.

3 Tổng quan về bộ dữ liệu

3.1 Đánh giá dữ liệu ban đầu

Tập dữ liệu gồm 100.000 quan sát và 28 thuộc tính, mô tả thông tin tài chính, hành vi tín dụng và một số thông tin cá nhân của khách hàng. Trường mục tiêu là cột phân loại điểm tín dụng (Credit Score) của từng cá nhân để đánh giá mức độ tin cậy của từng cụm khách hàng. Trong đó có 18 thuộc tính dạng số (numeric) và 10 thuộc tính dạng đối tượng (object)

Có thể nhận thấy rằng các trường Id, Customer_Id, Name, SSN và Type_Of_Loan có kiểu dữ liệu object và chủ yếu mang ý nghĩa định danh hoặc phục vụ cho mục đích truy vấn. Do đó, các trường này không đóng góp thông tin hữu ích cho mô hình học máy và sẽ được loại bỏ nhằm tối ưu hóa quá trình huấn luyện.

Ta có các chỉ số thống kê cho một số thuộc tính của bộ dữ liệu như sau:

	Monthly_Inhand_Salary	Num_Bank_Accounts	Num_Credit_Card	Interest_Rate	Delay_from_due_date	Num_Credit_Inquiries	Credit_Utilization_Ratio
count	84998.000000	100000.000000	100000.000000	100000.000000	100000.000000	98035.000000	100000.000000
mean	4194.170850	17.091280	22.47443	72.466040	21.068780	27.754251	32.285173
std	3183.686167	117.404834	129.05741	466.422621	14.860104	193.177339	5.116875
min	303.645417	-1.000000	0.000000	1.000000	-5.000000	0.000000	20.000000
25%	1625.568229	3.000000	4.000000	8.000000	10.000000	3.000000	28.052567
50%	3093.745000	6.000000	5.000000	13.000000	18.000000	6.000000	32.305784
75%	5957.448333	7.000000	7.000000	20.000000	28.000000	9.000000	36.496663
max	15204.633333	1798.000000	1499.000000	5797.000000	67.000000	2597.000000	50.000000

Hình 7: Thông số thống kê của một số thuộc tính trong bộ dữ liệu

Qua phân tích các chỉ số thống kê mô tả, tập dữ liệu cho thấy sự chênh lệch và phân tán lớn giữa các giá trị, đặc biệt ở các biến liên quan đến số lượng tài khoản ngân hàng, thẻ tín dụng và lãi suất. Một số trường xuất hiện giá trị bất

thường hoặc không hợp lệ (âm hoặc quá lớn), cần được xử lý trong giai đoạn tiền xử lý dữ liệu. Một số thuộc tính như `Monthly_Inhand_Salary` và `Total_EMI_per_month` có phân phối lệch phải, phản ánh sự tồn tại của một nhóm nhỏ khách hàng có thu nhập hoặc khoản vay rất cao. Ngược lại, `Credit_Utilization_Ratio` có giá trị ổn định và đáng tin cậy hơn. Nhìn chung, dữ liệu có nhiều dấu hiệu nhiễu và ngoại lai, do đó cần được làm sạch, chuẩn hoá và kiểm tra lại chất lượng trước khi đưa vào các mô hình học máy.

3.2 Tiền xử lý dữ liệu

3.2.1 Đối với dữ liệu số ta thực hiện các thao tác sau:

- Loại các kí tự không hợp lệ (vd: dấu _ ở trong số) và chuyển tất cả về số dương do các thuộc tính đều biểu diễn giá trị dương
- Thay thế các giá trị trống (NaN) bằng giá trị trung bình (mean) của cột đấy
- Tìm các giá trị ngoại lai dựa trên phương pháp tứ phân vị (IQR) để lọc ra 50% giá trị ở giữa của từng thuộc tính

3.2.2 Đối với dữ liệu category ta thực hiện các thao tác sau:

- Loại bỏ giá trị vô nghĩa (NaN, ___, chuỗi rỗng, ...)
- Mã hóa OneHot với phần còn lại
- Riêng với thuộc tính `Credit_History` đang có dạng chuỗi kí tự thể hiện thời gian (vd: “3 years and 2 months”) sẽ cần thiết kế hàm riêng để chuyển đổi về dữ liệu số

Kích thước dữ liệu giảm đi đáng kể chỉ còn 39917 bộ dữ liệu. Số thuộc tính tăng lên 49 do OneHot encoding tạo thêm cột mới ứng với từng nhãn của các cột category ban đầu.

Ta có các chỉ số thống kê của một số thuộc tính dạng số như sau:

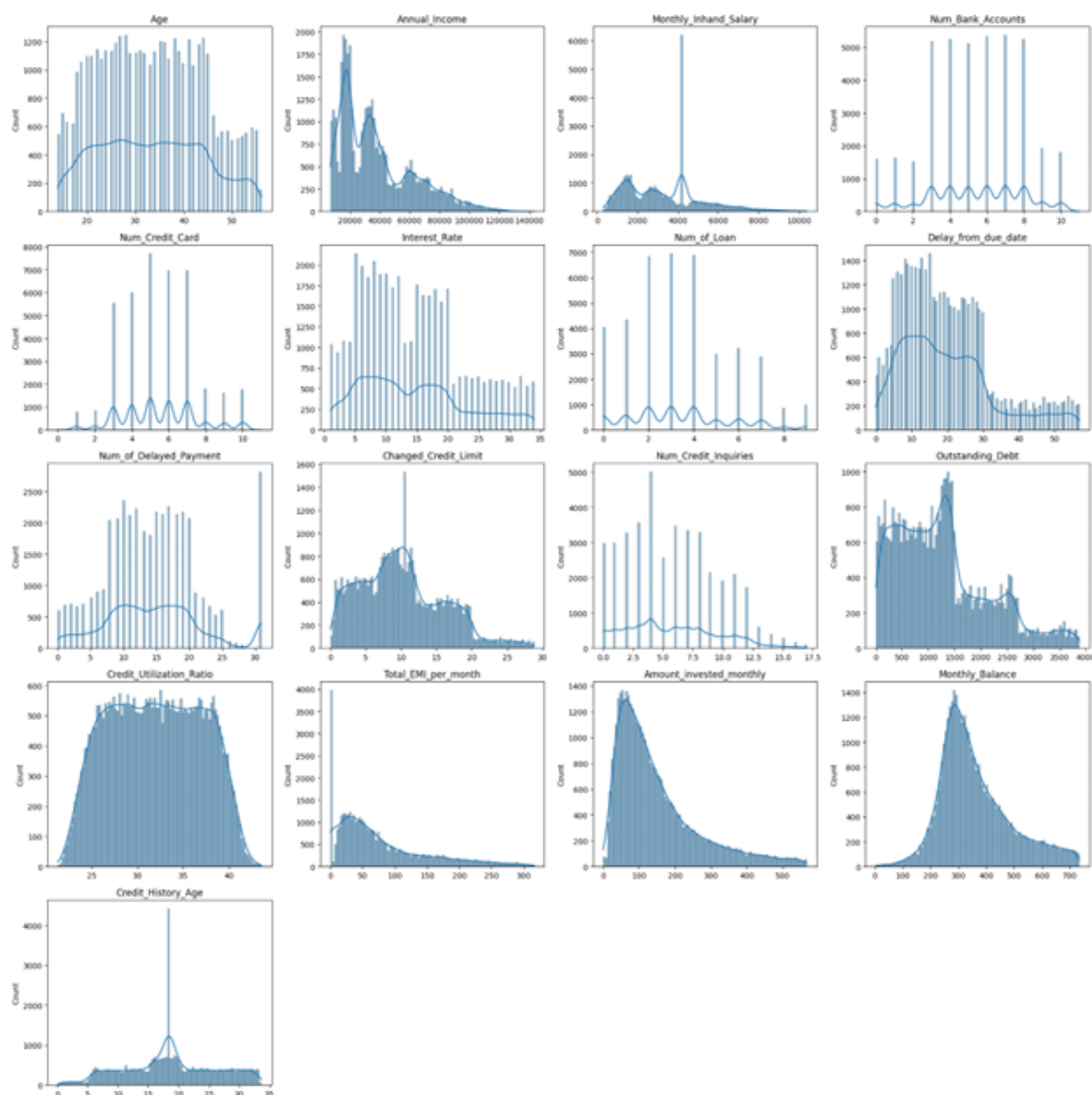
Monthly_Inhand_Salary	Num_Bank_Accounts	Num_Credit_Card	Interest_Rate	Num_of_Loan	Delay_from_due_date	Num_of_Delayed_Payment
39917.000000	39917.000000	39917.000000	39917.000000	39917.000000	39917.000000	39917.000000
3308.070528	5.347772	5.470802	14.298645	3.411579	19.837363	14.517602
1918.231715	2.504111	2.001660	8.504758	2.261386	13.097314	7.339552
303.645417	0.000000	0.000000	1.000000	0.000000	0.000000	0.000000
1645.192500	4.000000	4.000000	7.000000	2.000000	10.000000	9.000000
3031.179167	5.000000	5.000000	13.000000	3.000000	18.000000	14.000000
4194.170850	7.000000	7.000000	20.000000	5.000000	27.000000	19.000000
10389.015000	11.000000	11.000000	34.000000	9.000000	57.000000	30.946268

Hình 8: Chỉ số thống kê sau khi xử lý dữ liệu

Nhận xét: Sau khi thực hiện bước tiền xử lý, các giá trị thống kê của dữ liệu đã trở nên hợp lý và ổn định hơn so với ban đầu. Các giá trị âm hoặc bất thường đã được loại bỏ, phạm vi dữ liệu của các thuộc tính được thu hẹp và phù hợp hơn với thực tế.

3.3 Trực quan hóa dữ liệu

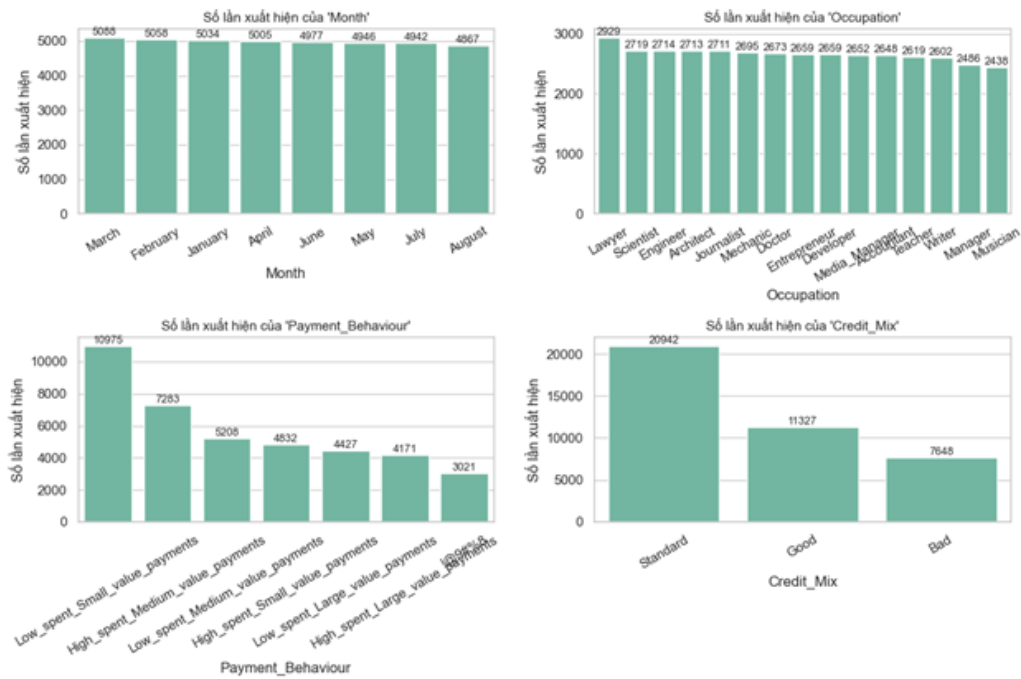
- Biểu đồ phân bố cho các thuộc tính số



Hình 9: Biểu đồ phân bố cho các thuộc tính số

Kết quả trực quan hoá cho thấy hầu hết các thuộc tính tài chính như Annual_Income, Monthly_Inhand_Salary, Total_EMI_per_month và Monthly_Balance có phân phối lệch phải, phản ánh việc phần lớn khách hàng có mức thu nhập và chi tiêu trung bình, trong khi chỉ một nhóm nhỏ có giá trị cao hơn đáng kể. Các thuộc tính rời rạc như Num_Bank_Accounts, Num_Credit_Card và Num_of_Loan phân bố tập trung ở các giá trị nhỏ (khoảng 3–7), thể hiện đặc điểm phổ thuộc tính trong hành vi tài chính. Các thuộc tính liên quan đến tín dụng (Interest_Rate, Delay_from_due_date, Credit_Utilization_Ratio) có phân phối hợp lý, mặc dù vẫn xuất hiện một số giá trị tập trung bất thường.

- Biểu đồ cột cho các thuộc tính Category



Hình 10: Biểu đồ phân bố các thuộc tính categorical

4 Giảm chiều dữ liệu (PCA & LDA)

Ta tiếp tục sử dụng phần dữ liệu đã qua xử lý ở trên, nhưng chỉ lấy ra 5000 mẫu ngẫu nhiên để thuận tiện cho việc trực quan.

Do cả 2 phương pháp giảm chiều đều phụ thuộc vào phương sai nên cần chuẩn hoá dữ liệu bằng thư viện `StandardScaler` để đảm bảo các biến đóng góp cân bằng và kết quả giảm chiều phản ánh đúng bản chất của dữ liệu.

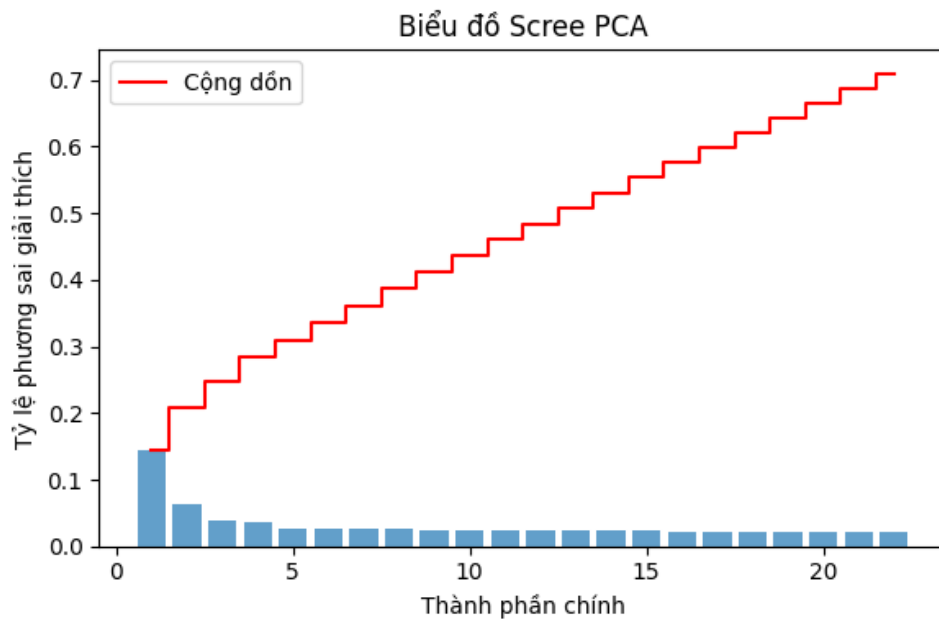
4.1 Phân tích thành phần chính (PCA)

Với mức phương sai giữ lại là 70% ta thu được 22 PC với tỉ lệ phương sai giải thích tương ứng như sau:

Tỷ lệ phương sai giải thích:

```
[0.1444 0.0639 0.039 0.0363 0.0264 0.0261 0.0255 0.0251 0.0249 0.0247
 0.024 0.0238 0.0236 0.0233 0.0231 0.0225 0.0224 0.0224 0.0223 0.0222
 0.022 0.0219]
```

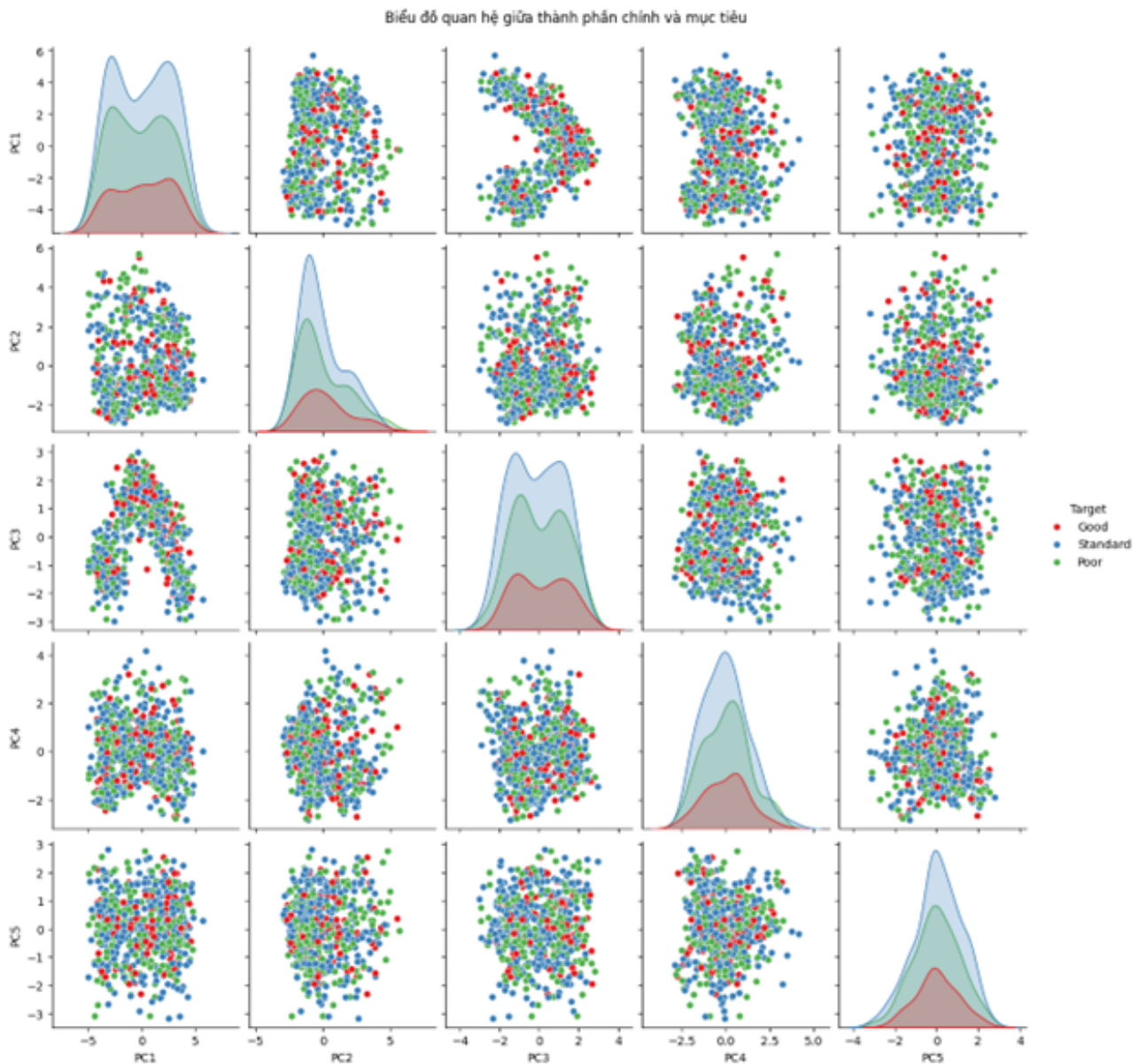
Tổng phương sai giữ lại: 0.7099



Hình 11: Biểu đồ Screeplot cho 22 PC

Nhận xét: Không có thành phần chính nào thực sự chiếm ưu thế trong việc giải thích phương sai của dữ liệu. Trong đó thành phần chính đầu tiên (PC1) chứa phần thông tin lớn nhất nhưng không quá vượt trội để đặc trưng cho bộ dữ liệu. Các thành phần còn lại chủ yếu phản ánh nhiễu hoặc các yếu tố phụ, ít đóng góp vào việc mô tả tổng thể tập dữ liệu.

*)Biểu diễn trực quan bằng biểu đồ mối quan hệ giữa các thành phần chính (PCs) sau khi giảm chiều bằng PCA và nhận mục tiêu. Với mục tiêu của việc vẽ biểu đồ này là đánh giá khả năng phân tách dữ liệu trong không gian mới do PCA tạo ra.



Hình 12: Biểu đồ tương quan giữa thành phần chính và mục tiêu

Nhận xét: Các nhóm mục tiêu phân bố khá chồng lấn nhau trên hầu hết các trục PC, không hình thành được ranh giới hay cụm rõ rệt. Cho thấy việc giảm chiều không mang lại nhiều lợi ích thực tế cho mô hình phân loại, vì phương sai dữ liệu được phân tán khá đồng đều và không tập trung mạnh ở một vài hướng chính.

4.2 Phân tích biệt thức tuyến tính (LDA)

Vì trong bài này biến mục tiêu có 3 lớp (“Good”, “Standard”, “Poor”), nên LDA chỉ có thể tạo ra tối đa 2 thành phần phân biệt ($3 - 1 = 2$). Hai trục này thể hiện 2 hướng giúp tách biệt các nhóm tốt nhất trong không gian dữ liệu.

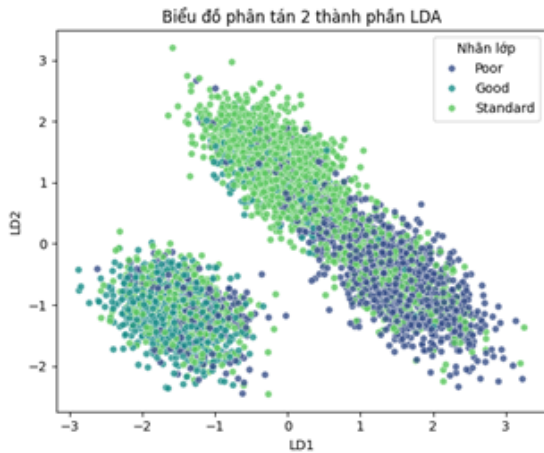
Kết quả thu được của việc giảm chiều LDA:

Phân tích LDA trích xuất được hai thành phần phân biệt chính với các giá trị riêng lần lượt là 0.6183 và 0.3817, cho thấy sự giải thích tương đối tốt cho dữ liệu ban đầu. Các biến Annual_Income, Interest_Rate, và Num_Credit_Card có trọng số cao nhất ở thành phần đầu tiên, trong khi Annual_Income, Changed_Credit_Limit, và Interest_Rate nổi bật ở thành phần thứ hai. Kết quả cho thấy thu nhập và lãi suất là những yếu tố có ảnh hưởng mạnh nhất trong việc phân biệt

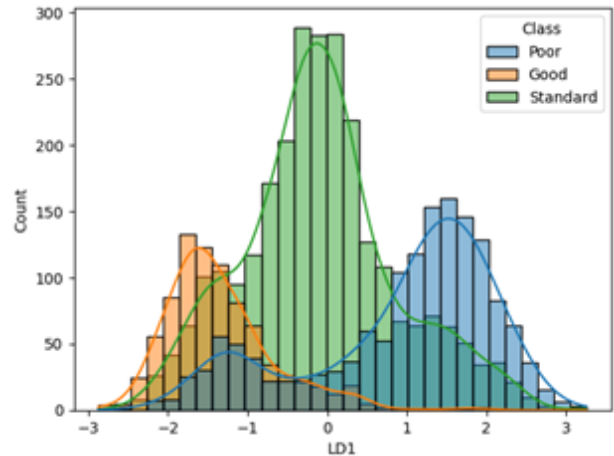
các nhóm khách hàng.

Một số mô hình để trực quan đánh giá chức năng của LDA lên bộ dữ liệu:

- Mô hình phân tán (Scatter plot)
- Mô hình phân bố (Histogram plot)



(a) Biểu đồ phân tán



(b) Biểu đồ phân bố

Hình 13: Biểu đồ phân tán và biểu đồ phân bố 2 thành phần của LDA

Nhận xét:

- Ba nhóm khách hàng (“Poor”, “Good”, “Standard”) được phân tách tương đối rõ ràng trong không gian hai chiều LD1 và LD2. Nhóm “Poor” tập trung ở phía bên phải đồ thị ($LD1 > 0$), trong khi nhóm “Good” nằm chủ yếu ở vùng LD1 âm. Nhóm “Standard” phân bố ở giữa, có sự giao thoa nhẹ với hai nhóm còn lại. Điều này cho thấy hai thành phần LDA đã mô tả khá tốt sự khác biệt giữa các nhóm.
- Nhìn chung, kết quả cho thấy mô hình LDA có khả năng phân biệt tốt giữa các nhóm khách hàng dựa trên các đặc trưng đầu vào, đồng thời hai thành phần LD1 và LD2 giữ lại phần lớn thông tin giúp tách biệt các nhóm.

5 Phân cụm dữ liệu (Thuật toán K-Means)

5.1 Cách chọn số cụm K

Trong thuật toán K-Means, số cụm K cần được xác định trước khi tiến hành phân cụm. Việc lựa chọn giá trị K phù hợp có vai trò quan trọng, vì nếu chọn K quá nhỏ sẽ khiến các cụm bị gộp chung và mất thông tin chi tiết, trong khi chọn K quá lớn lại gây phân mảnh dữ liệu, làm giảm khả năng khái quát hóa.

Ở đây ta kết hợp trực quan đồ thị với Elbow method và Silhouette Score để quyết định số cụm tối ưu cho việc phân cụm dữ liệu:

5.2 Xác định số cụm tối ưu

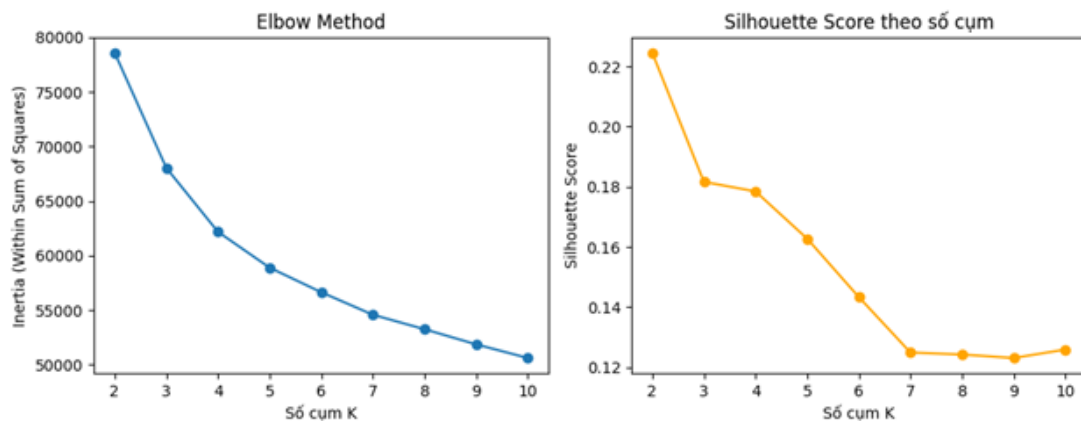
- Elbow method:

Dựa trên việc quan sát sự thay đổi của chỉ số Inertia (Within-Cluster Sum of Squares – WCSS) theo số cụm KKK. Khi K tăng, giá trị Inertia giảm dần, nhưng đến một điểm sẽ giảm chậm lại – tạo thành “khuỷu tay” trên biểu đồ.

- Phương pháp Silhouette Score:

Đánh giá chất lượng phân cụm dựa trên mức độ gắn kết của các điểm dữ liệu trong cùng một cụm và mức độ tách biệt giữa các cụm khác nhau.

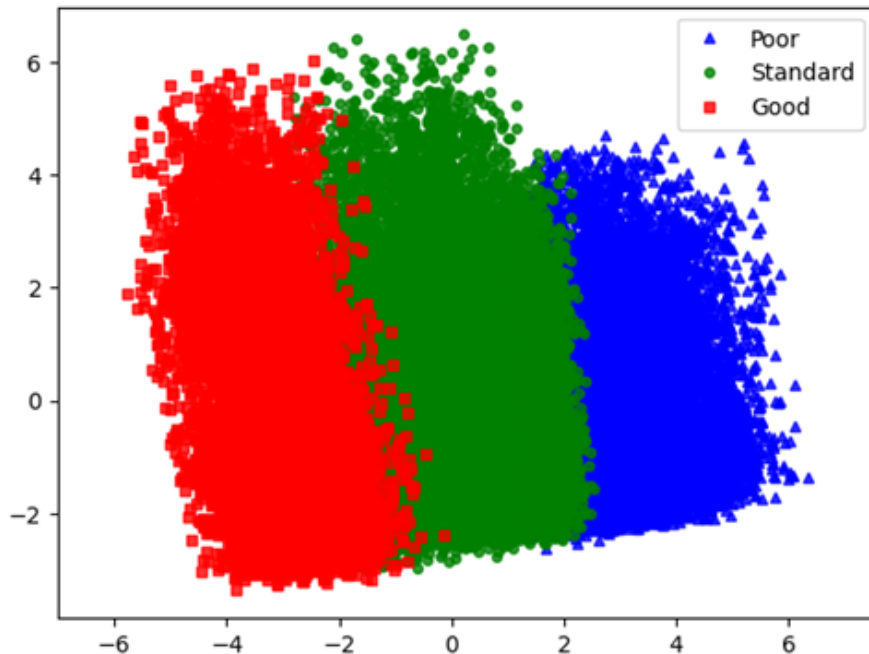
Giá trị Silhouette Score dao động trong khoảng $[-1, 1]$, trong đó giá trị càng cao thể hiện cấu trúc cụm càng rõ ràng. Số cụm K tối ưu là giá trị tại đó Silhouette Score đạt cực đại.



Hình 14: Đồ thị đánh giá mức tối ưu số cụm k

5.3 Phân cụm với K=3

Dựa trên phương pháp Elbow, giá trị WCSS giảm mạnh đến $K = 3$ và sau đó giảm chậm dần, cho thấy việc tăng số cụm không còn mang lại cải thiện đáng kể. Đồng thời, Silhouette Score đạt giá trị tương đối cao tại $K = 3$ và giảm khi K tăng, phản ánh cấu trúc phân cụm rõ ràng nhất tại giá trị này. Vì vậy, $K = 3$ được lựa chọn là số cụm phù hợp cho bài toán.



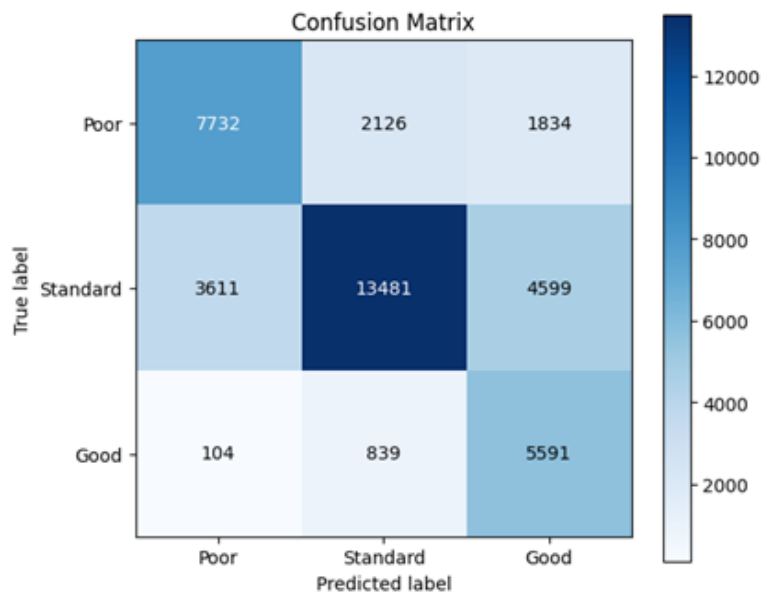
Hình 15: Trực quan hóa kết quả phân cụm với $K=3$

```
Cluster → Label mapping: {0: np.int64(0), 1: np.int64(2), 2: np.int64(1)}  
Accuracy : 0.6714933486985495  
Silhouette Score: 0.3210439787750204
```

Sau khi thực hiện K-means với $K = 3$, nhãn cụm được ánh xạ sang nhãn thực tế bằng phương pháp mode.

Độ chính xác (Accuracy) thu được: 0.67, chỉ dùng để tham khảo không phải mục tiêu tối ưu của phân cụm. Giá trị Silhouette trung bình thu được là 0.32

Mặc dù K-means là thuật toán học không giám sát và không tối ưu trực tiếp cho accuracy, kết quả này cho thấy mức độ tương đồng tương đối giữa các cụm và nhãn tín dụng thực tế. Các cụm có sự phân tách ở mức trung bình, dữ liệu không hoàn toàn tách biệt tuyến tính



Hình 16: Ma trận nhầm lẫn

Ma trận nhầm lẫn

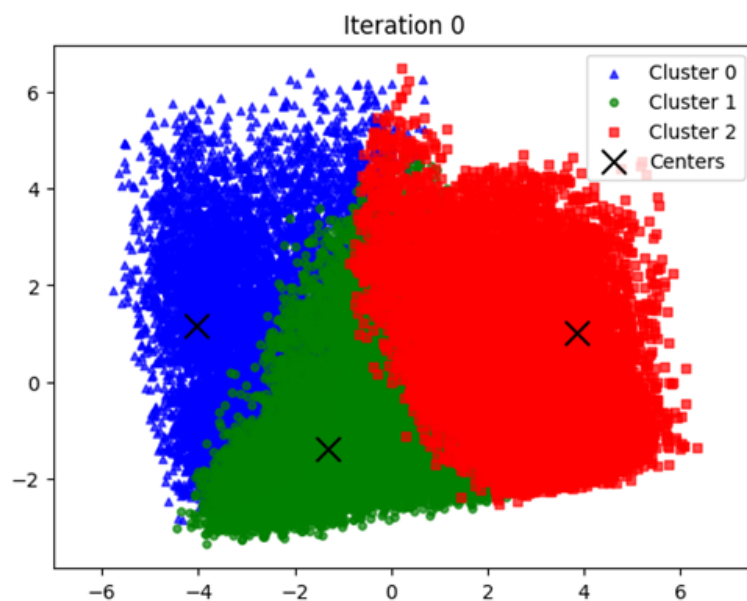
- Các lớp Poor và Good có mức độ tách biệt tốt hơn
- Lớp Standard có sự chồng lấn lớn, dẫn đến nhiều trường hợp nhầm lẫn

Việc trực quan hóa cho thấy cấu trúc cụm tương đối rõ ràng, tuy nhiên vẫn tồn tại vùng chồng lấn giữa các cụm, đặc biệt là ở lớp Standard.

Trực quan quá trình phân cụm K-Means

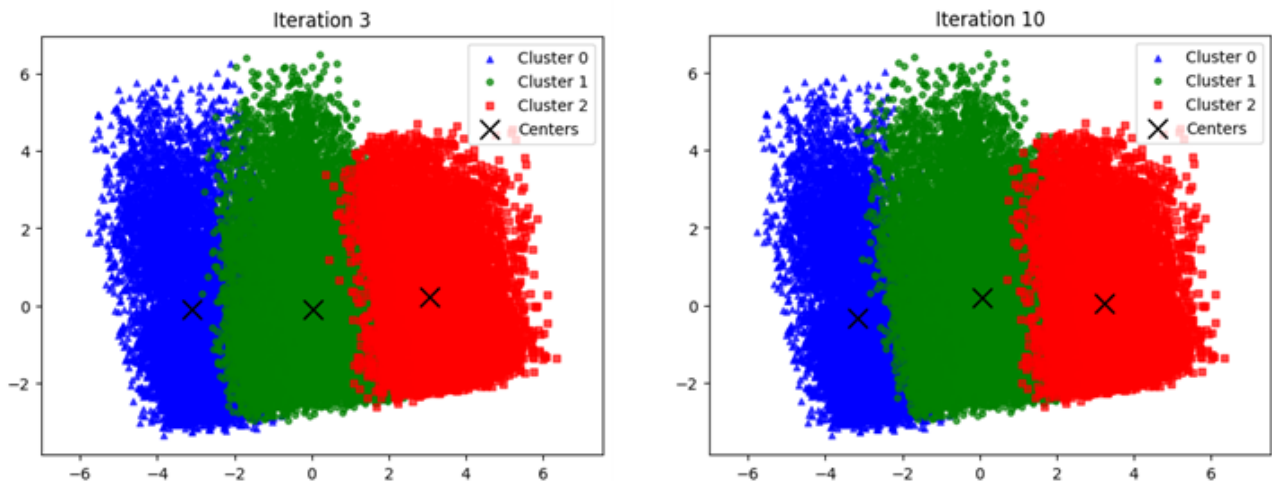
Quá trình hội tụ của thuật toán được minh họa thông qua trực quan hóa 2D sau mỗi lần cập nhật tâm cụm.

Iteration 0: các tâm cụm được khởi tạo, phân bố còn lộn xộn



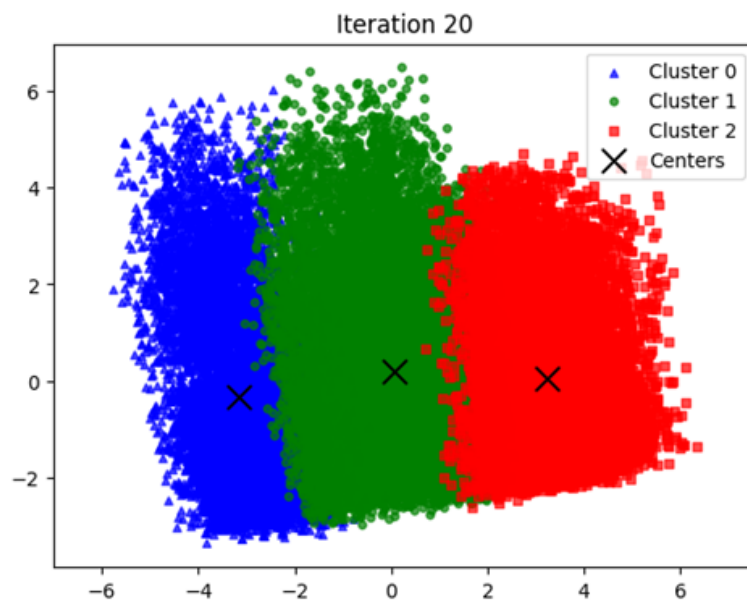
Hình 17: Iteration 0

Iteration 3 - 10: các tâm cụm dần dịch chuyển về vùng mật độ cao



Hình 18: Iteration 3 - 10

Iteration 20 (Kết quả cuối): các tâm cụm ổn định, không còn thay đổi đáng kể



Hình 19: Iteration 20

Điều này cho thấy thuật toán K-means đã hội tụ sau một số vòng lặp nhất định.

5.3.1 Kết Luận

Trong phần này, thuật toán K-means đã được áp dụng thành công để phân cụm dữ liệu khách hàng tín dụng. Kết quả cho thấy:

- Các cụm thu được có mức độ tương đồng tương đối với nhân tín dụng thực tế
- Dữ liệu tồn tại sự chồng lấn giữa các lớp, đặc biệt là lớp Standard

- K-means phù hợp cho việc khám phá cấu trúc dữ liệu

Việc kết hợp PCA với K-Means giúp trực quan hóa dữ liệu tốt hơn trong không gian giảm chiều. Tuy nhiên, quá trình giảm chiều có thể làm mất một phần thông tin quan trọng, khiến K-Means khó phân tách các cụm một cách chính xác. Giá trị Silhouette thấp cùng với chỉ số ARI (Adjusted Rand Index) thấp cho thấy cấu trúc dữ liệu có xu hướng liên tục hơn là phân nhóm rời rạc.

5.4 Phân cụm dữ liệu - GMM

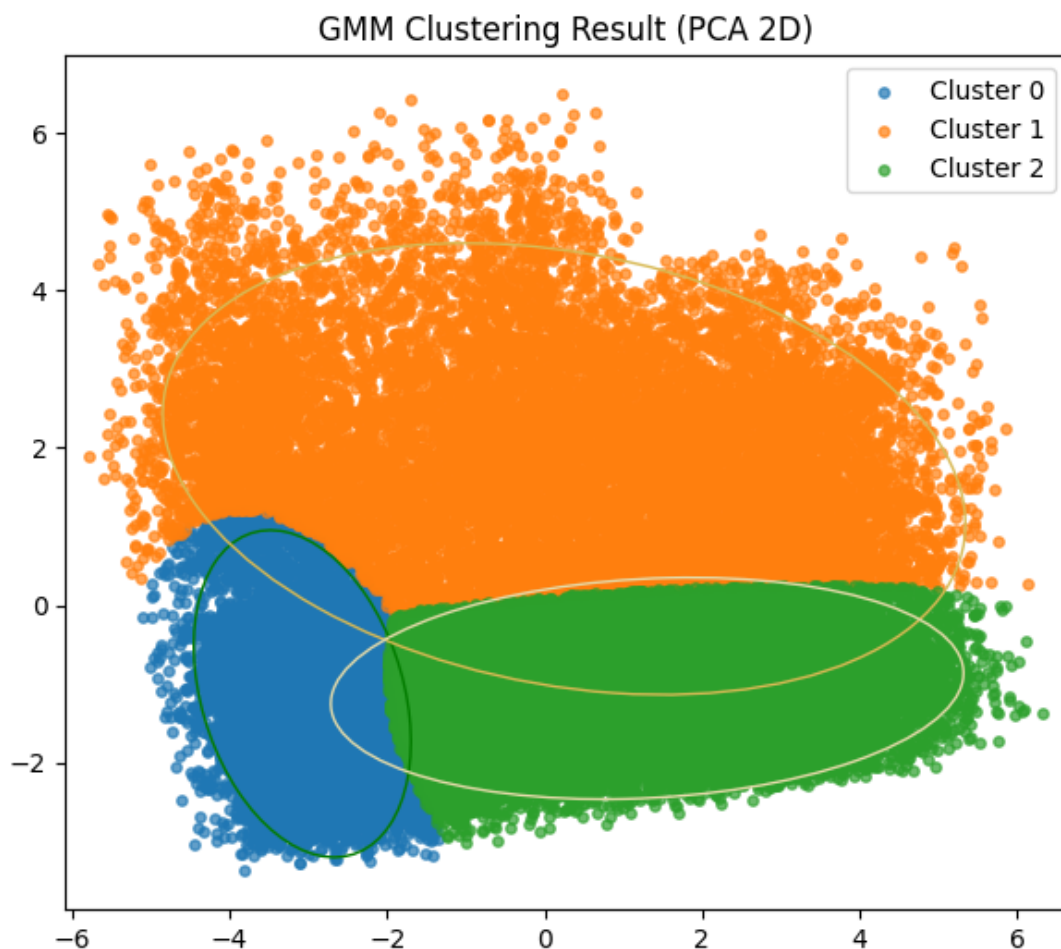
5.4.1 Thực hiện phân cụm GMM

Phương pháp Gaussian Mixture Model (GMM) được sử dụng để phân cụm dữ liệu với số cụm $K = 3$, tương ứng với ba mức đánh giá của biến Credit_Score.

Mô hình GMM được huấn luyện bằng thuật toán Expectation–Maximization (EM), trong đó các tham số của từng Gaussian (trung bình, ma trận hiệp phương sai và trọng số cụm) được cập nhật lặp cho đến khi mô hình đạt trạng thái ổn định theo số vòng lặp đã thiết lập.

Sau quá trình huấn luyện, mỗi mẫu dữ liệu được gán vào cụm có xác suất thuộc cụm lớn nhất.

5.4.2 Kết quả phân cụm



Hình 20: Phân cụm GMM

Kết quả phân cụm bằng Gaussian Mixture Model cho thấy các cụm dữ liệu có dạng elip, phù hợp với giả định phân bố Gaussian của mô hình. Các cụm có mức độ phân tán và hướng phân bố khác nhau, phản ánh sự không đồng nhất của dữ liệu đầu vào.

Một số vùng dữ liệu xuất hiện sự chồng lấn giữa các cụm, cho thấy ranh giới giữa các nhóm không hoàn toàn tách biệt rõ ràng, đặc biệt đối với các mẫu dữ liệu nằm gần vùng biên.

Sau khi ánh xạ các cụm bằng phương pháp bỏ phiếu đa số, các cụm GMM thể hiện sự tương ứng tương đối với các nhãn Poor, Standard và Good.

5.4.3 Kết luận

Nhìn chung, GMM mô hình hóa tốt cấu trúc xác suất của dữ liệu và cung cấp kết quả phân cụm hợp lý trong bối cảnh dữ liệu có sự giao thoa giữa các nhóm.

Phương pháp GMM cho kết quả phân cụm hợp lý, phản ánh được cấu trúc xác suất của dữ liệu và cung cấp góc nhìn bổ sung so với các phương pháp phân cụm cứng như K-means.

6 Đánh giá mô hình K-Nearest Neighbors

6.1 Phương pháp đánh giá

6.1.1 Phân chia dữ liệu

Các tỷ lệ train:test được thử nghiệm: [8:2, 7:3, 6:4, 9:1]

Việc thử nghiệm nhiều tỷ lệ giúp đánh giá độ ổn định và khả năng tổng quát hóa của mô hình K-NN.

6.1.2 Lựa chọn tham số K

K ban đầu: $k = \sqrt{n}$ (căn bậc 2 số mẫu dữ liệu training)

K tối ưu: Sử dụng phương pháp k-fold Cross Validation

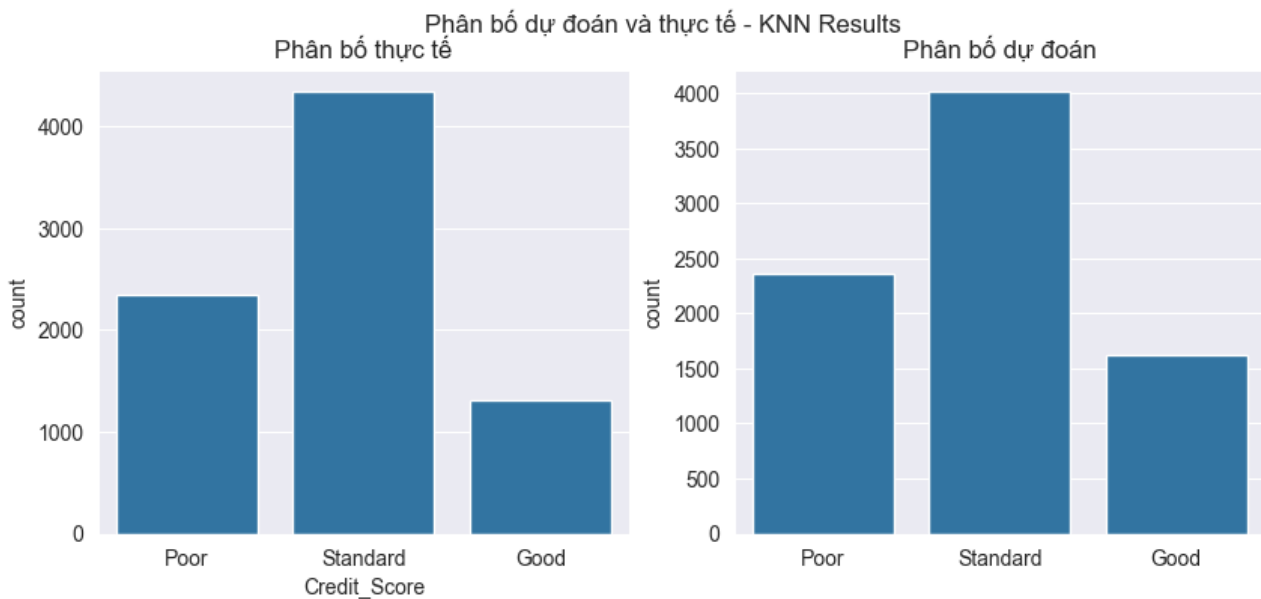
- Để xác định giá trị k phù hợp nhất, tiến hành Cross-Validation (5-fold) nhằm đánh giá độ chính xác trung bình của mô hình với các giá trị k khác nhau
- Các giá trị k trong khoảng $[k - 10; k + 10]$ với $k = \sqrt{n_{train}}$
- Với mỗi giá trị k , mô hình K-NN được huấn luyện và đánh giá bằng hàm `cross_val_score()` trên 5 tập chia dữ liệu (5-fold CV)
- Giá trị k tốt nhất (`best_k`) là giá trị cho độ chính xác trung bình cao nhất trên toàn bộ các lần kiểm định chéo

Bảng 1: Độ chính xác (Accuracy) theo các Phương pháp Tiền xử lý

Loại dữ liệu	$K = \sqrt{n} = 178$			K tối ưu = 175			Nhận xét
	Acc Train	Acc Test	Chênh lệch	Acc Train	Acc Test	Chênh lệch	
Gốc	62.29	61.91	0.38	62.34	61.71	0.63	Không bị overfitting (0.38–0.63)
PCA	67.03	66.43	0.60	67.06	66.47	0.59	
LDA	70.63	70.04	0.59	70.63	70.00	0.63	

6.2 Kết quả thực nghiệm

6.2.1 Tỷ lệ train:test = 80:20



Hình 21: Phân bố thực tế và dự đoán

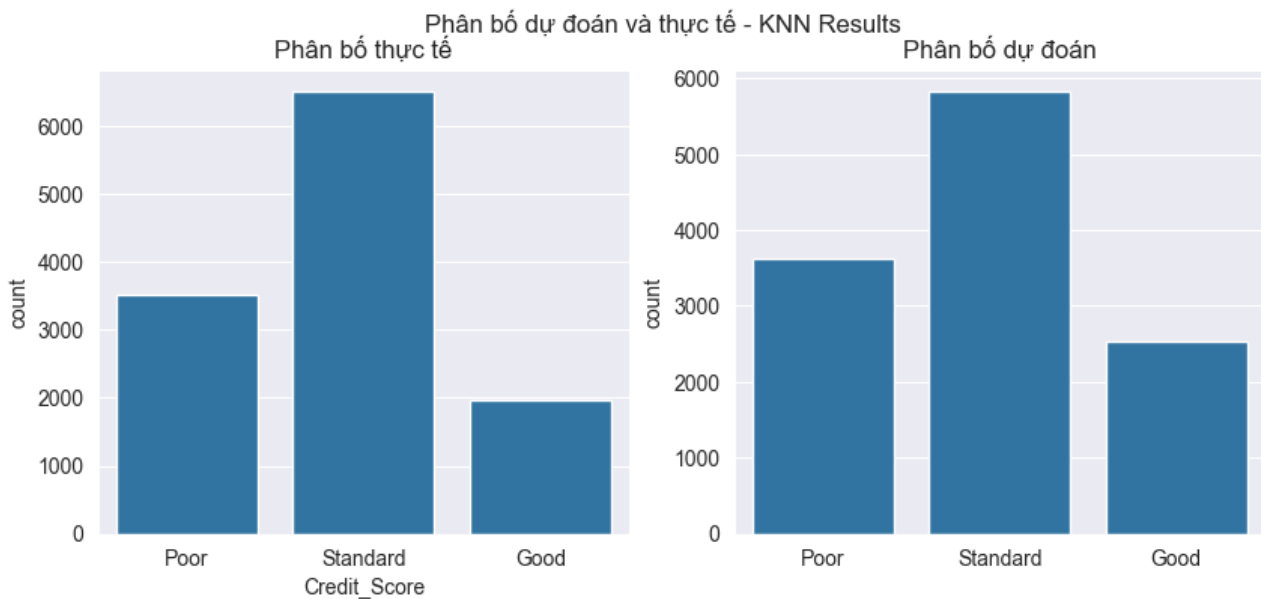
Phân tích kết quả:

- Ma trận: Dự đoán có xu hướng 'Standard' tỷ lệ đúng 77.14% và tệ nhất ở lớp Good tỷ lệ sai 45.27
- Phân bố: Dự đoán gần với thực tế, nhưng mô hình hơi thiên vị dự đoán 'Good' nhiều hơn dẫn đến over-prediction và 'Standard' bị under-predicted nhẹ

Bảng 2: Kết quả thực nghiệm với tỷ lệ train:test = 70:30

Loại dữ liệu	$K = \sqrt{n} = 167$			K tối ưu = 157			Nhận xét
	Acc Train	Acc Test	Chênh lệch	Acc Train	Acc Test	Chênh lệch	
Gốc	62.36	61.68	0.68	62.34	61.79	0.55	Không bị overfitting (0.68-1.05)
PCA	66.93	66.11	0.82	67.03	66.07	0.96	
LDA	70.71	69.76	0.95	70.73	69.68	1.05	

6.2.2 Tỷ lệ train:test = 70:30



Hình 22: Phân bố thực tế và dự đoán

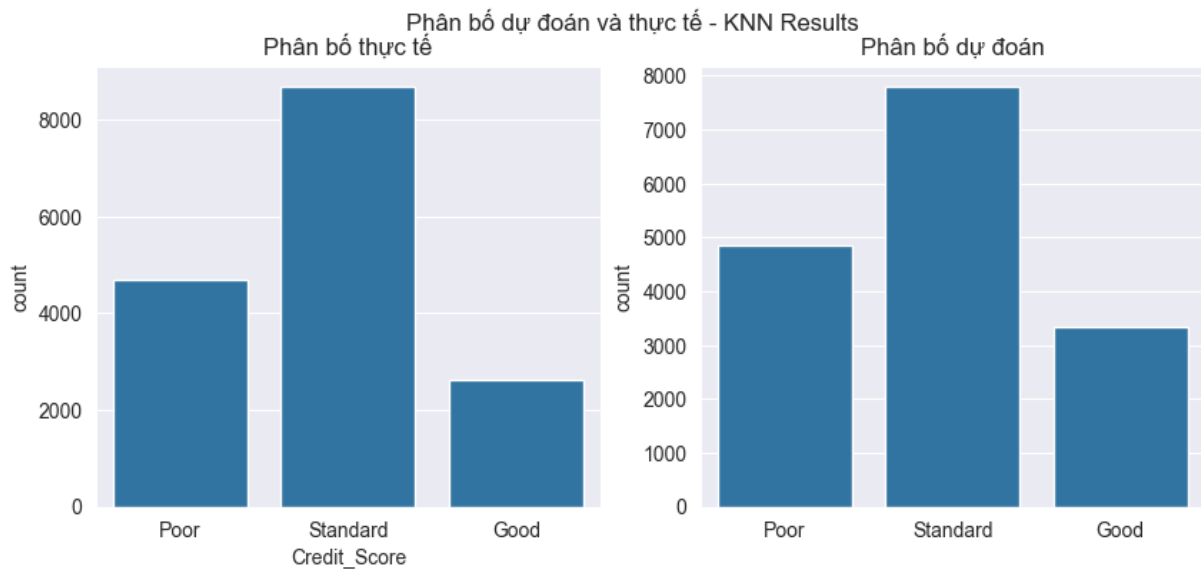
Phân tích kết quả:

- Xu hướng nhầm lẫn tương tự tỷ lệ 80:20, lớp Good vẫn khó phân loại nhất, lớp Good và lớp Poor chủ yếu bị nhầm thành Standard
- Phân bố vẫn giữ nguyên hình dạng như tỷ lệ 80:20, nhưng đã gần khớp hơn nhờ tập test lớn hơn

6.2.3 Tỷ lệ train:test = 60:40

Bảng 3: Kết quả thực nghiệm với tỷ lệ train:test = 60:40

Loại dữ liệu	$K = \sqrt{n} = 154$			K tối ưu = 152			Nhận xét
	Acc Train	Acc Test	Chênh lệch	Acc Train	Acc Test	Chênh lệch	
Gốc	62.29	61.55	0.74	62.28	61.59	0.69	Không bị overfitting (0.68-1.08)
PCA	67.09	66.41	0.68	67.05	66.32	0.73	
LDA	70.62	69.65	0.97	70.65	69.57	1.08	



Hình 23: Phân bố thực tế và dự đoán

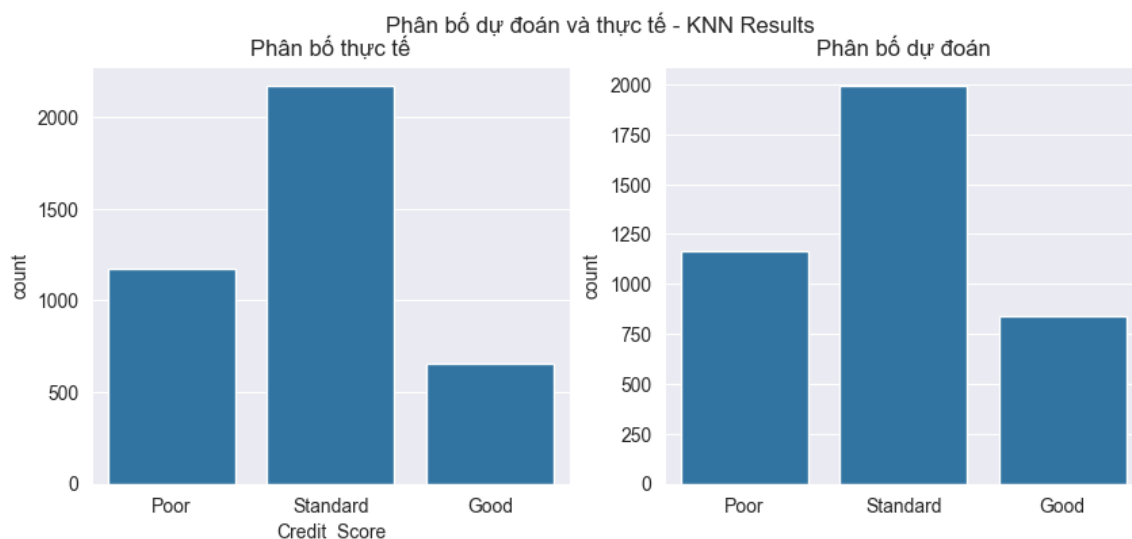
Phân tích kết quả:

- Dự đoán đúng Good giảm đi nhưng xu hướng vẫn nhất quán
- • Over-predict 'Good' rõ rệt hơn có thể do tập test lớn

6.2.4 Tỷ lệ train:test = 90:10

Bảng 4: Kết quả thực nghiệm với tỷ lệ train:test = 90:10

Loại dữ liệu	$K = \sqrt{n} = 189$			K tối ưu = 183			Nhận xét
	Acc Train	Acc Test	Chênh lệch	Acc Train	Acc Test	Chênh lệch	
Gốc	62.27	62.45	-0.18	62.29	61.47	0.82	Không bị overfitting (-0.18-1.16)
PCA	66.98	66.36	0.62	66.96	66.18	0.78	
LDA	70.72	69.59	1.13	70.75	69.59	1.16	



Hình 24: Phân bố thực tế và dự đoán

Phân tích kết quả:

- Ít lỗi hơn do test nhỏ nhưng predict 'Good' vẫn rất thấp 53.7% gần như không bị
- Lớp Good bị over-predict, lớp Poor có dự đoán gần nhất với thực tế

6.3 Kết luận về mô hình K-NN

6.3.1 Đánh giá về Accuracy

- Accuracy dao động trong khoảng 61.5% - 62.3% (với dữ liệu Gốc) và 69-70% (với dữ liệu LDA), là mức tương đối cho mô hình K-NN
- Chênh lệch giữa train và test < 1.2%, cho thấy độ ổn định tương đối, không bị overfitting

6.3.2 Đánh giá theo nhóm dữ liệu

- **Độ chính xác:** LDA > PCA > Gốc
- **Dữ liệu gốc:** cho độ chênh lệch và độ chính xác thấp nhất do còn đầy đủ thông tin đặc trưng nhưng sẽ bị nhiễu do phân bố đặc trưng không cân bằng
- **Dữ liệu LDA:** cho độ chính xác tốt nhất và ổn định, tổng quát hóa tốt
- **Dữ liệu PCA:** cho độ chính xác trung bình, cải thiện hơn dữ liệu gốc

6.3.3 Đánh giá theo tỷ lệ train:test

- Train:Test tỷ lệ 80:20 cho kết quả tốt nhất
- Tỷ lệ 90:10 do có quá nhiều dữ liệu train dẫn đến overfitting và giảm performance
- Tỷ lệ 70:30 và 60:40 cho kết quả tương đương nhau
- **Kết luận:** Tỷ lệ 80:20 là phù hợp nhất

6.3.4 Đánh giá về tham số K

- K ban đầu cho độ chênh lệch train-test thấp hơn nhưng K tối ưu cho độ chính xác tốt hơn nhưng không đáng kể, nhìn chung là tương đối nên K ban đầu khá ổn định
- K ban đầu có giá trị cao hơn nên over-smoothing nên K tối ưu giảm vài đơn vị ở trong tất cả các tỷ lệ

6.3.5 Cấu hình tốt nhất

Thông số tối ưu:

- Phương pháp: LDA
- Train/Test: 80:20
- $K = 175$
- Accuracy: 70.04
- Overfitting: 0.63

Lý do: LDA giảm chiều xuống 2 components nhưng giữ được thông tin phân biệt lớp tốt nhất

6.3.6 Hạn chế

- Mô hình chưa tối ưu trong việc phân biệt các lớp
- Lớp Good và Poor có xu hướng nhầm lẫn thành Standard do phân bố thực tế lớp Standard chiếm ưu thế lớn nhất dẫn đến những nhầm lẫn các mẫu thiếu số thành lớp đa số

6.3.7 Độ phù hợp của K-NN

Độ phù hợp của K-NN cho bài toán ở mức cơ bản, K-NN có hiệu suất ổn định và nhất quán, là mô hình tham chiếu (baseline) tốt cho bài toán. Tuy nhiên, để tối ưu hiệu quả và an toàn hơn cần phải áp dụng các kỹ thuật tiền xử lý cân bằng dữ liệu hoặc sử dụng mô hình mạnh hơn để cải thiện khả năng phân loại lớp thiểu số.

7 Đánh giá mô hình Softmax Regression

7.1 PHƯƠNG PHÁP ĐÁNH GIÁ

7.1.1 Phân chia dữ liệu

Các tỷ lệ **train:test** được thử nghiệm gồm:

- 80:20
- 70:30
- 60:40

Việc thử nghiệm nhiều tỷ lệ giúp đánh giá khả năng **tổng quát hóa** và mức độ **ổn định** của mô hình. Với mỗi tỷ lệ, dữ liệu được chuẩn hóa, huấn luyện, sau đó đánh giá riêng trên tập test.

7.1.2 Lựa chọn tham số và chuẩn hóa dữ liệu

Trước khi huấn luyện, toàn bộ dữ liệu được **chuẩn hóa theo StandardScaler**, đưa mỗi đặc trưng về phân phối có **trung bình 0** và **độ lệch chuẩn 1**.

Điều này giúp các hệ số θ_k hội tụ ổn định hơn.

Mô hình được huấn luyện với **solver 'lbfgs'** (tối ưu hóa tốt cho đa lớp) và tham số **multi_class='multinomial'**.

7.2 THỰC NGHIỆM

7.2.1 Quy trình huấn luyện

Quy trình được thực hiện như sau:

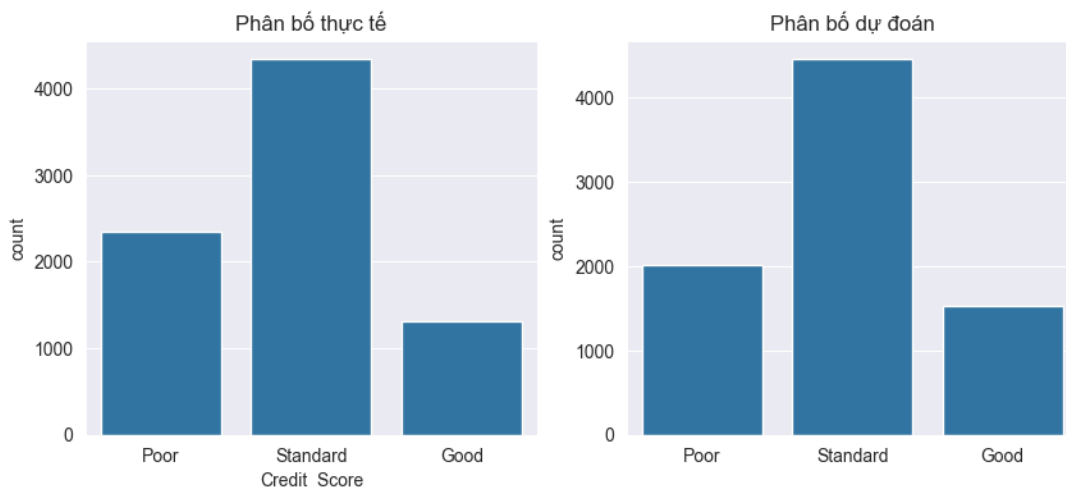
1. Chuẩn hóa dữ liệu gốc.
2. Giảm chiều dữ liệu bằng **PCA** và **LDA** để so sánh hiệu quả.
3. Huấn luyện mô hình Softmax Regression trên 3 bộ dữ liệu: gốc, PCA, LDA.
4. Đánh giá mô hình trên tập test tương ứng.
5. Ghi nhận các giá trị **độ chính xác** và **sai lệch** giữa train-test.

7.2.2 Kết quả thực nghiệm

Tỷ lệ train:test = 80:20

Bảng 5: Kết quả đánh giá mô hình với tỷ lệ 80:20

Loại dữ liệu	Accuracy Train	Accuracy Test	Chênh lệch
Gốc	65.12%	64.85%	0.27%
PCA	69.43%	68.90%	0.53%
LDA	72.01%	71.56%	0.45%



Hình 25: Kết quả tỉ lệ 8:2

Nhận xét:

- **LDA** đạt kết quả tốt nhất, thể hiện khả năng giữ thông tin phân biệt lớp cao.
- Dữ liệu **PCA** cũng cải thiện đáng kể so với dữ liệu gốc, chứng tỏ việc giảm chiều giúp loại bỏ nhiễu.
- Chênh lệch giữa train và test đều nhỏ hơn 1%, mô hình ổn định và **không overfitting**.

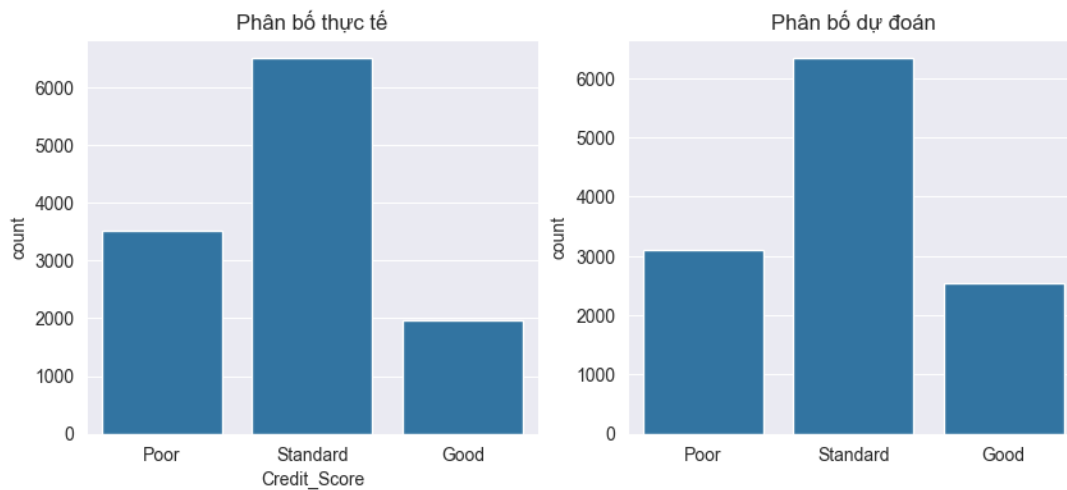
Tỷ lệ train:test = 70:30

Nhận xét:

- Kết quả nhất quán với tỷ lệ 80 : 20.
- Dữ liệu **LDA** tiếp tục cho hiệu suất cao nhất.
- Với tập test lớn hơn, độ lệch train-test tăng nhẹ, phản ánh khả năng tổng quát hóa thực tế hơn.

Bảng 6: Kết quả đánh giá mô hình với tỷ lệ 70:30

Loại dữ liệu	Accuracy Train	Accuracy Test	Chênh lệch
Gốc	64.91%	64.10%	0.81%
PCA	69.07%	68.44%	0.63%
LDA	71.83%	71.04%	0.79%

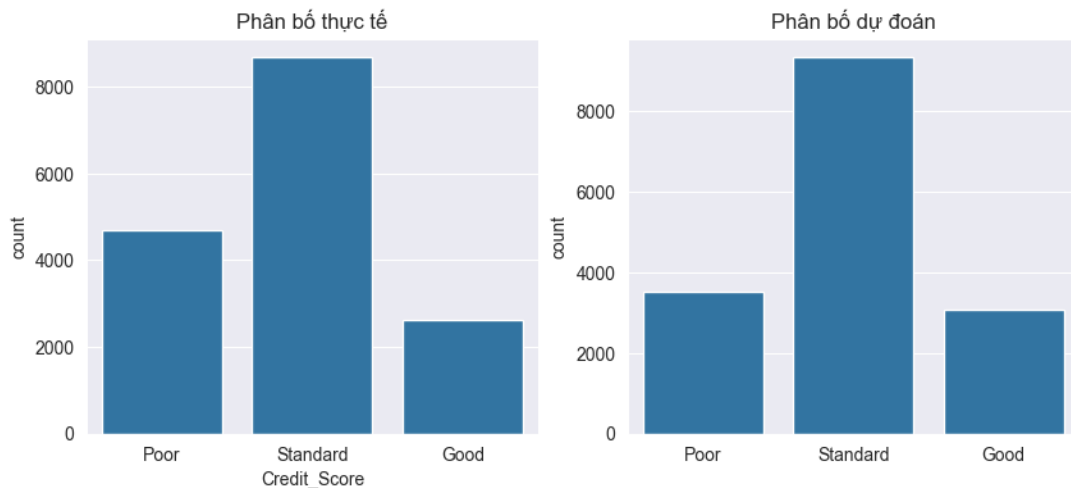


Hình 26: Kết quả tỉ lệ 7:3

Bảng 7: Kết quả đánh giá mô hình với tỷ lệ 60:40

Loại dữ liệu	Accuracy Train	Accuracy Test	Chênh lệch
Gốc	64.57%	63.94%	0.63%
PCA	68.70%	68.18%	0.52%
LDA	71.45%	70.86%	0.59%

Tỷ lệ train:test = 60:40



Hình 27: Kết quả tỉ lệ 6:4

Nhận xét:

- **Accuracy** giảm nhẹ do tập huấn luyện nhỏ hơn, nhưng vẫn giữ được tính ổn định.
- Dữ liệu **PCA** và **LDA** tiếp tục duy trì ưu thế rõ rệt.
- Độ lệch dưới 1% cho thấy mô hình **không overfit**, hồi tụ tốt.

7.3 KẾT LUẬN

7.3.1 Đánh giá về độ chính xác (Accuracy)

Kết quả thực nghiệm cho thấy mô hình Softmax Regression đạt độ chính xác trung bình:

- Dữ liệu **Gốc**: khoảng 63 – 65%,
- Dữ liệu sau **PCA**: khoảng 67 – 68%,
- Dữ liệu sau **LDA**: đạt cao nhất, khoảng **70 – 72%**.

Điều này cho thấy việc giảm chiều bằng LDA giúp tăng khả năng phân biệt giữa các lớp điểm tín dụng (*Poor – Standard – Good*), do LDA hướng tới tối đa hóa khả năng tách biệt tuyến tính giữa các nhãn.

Bảng 8: Đánh giá hiệu suất theo nhóm dữ liệu

Loại dữ liệu	Accuracy TB	Chênh lệch	Nhận xét
Gốc (Original)	63 – 65%	1 – 2%	Ổn định, ít nhiễu nhưng độ chính xác thấp hơn.
PCA (Giữ 70% phương sai)	67 – 68%	2 – 3%	Cải thiện hiệu suất, giảm nhiễu nhưng mất một phần thông tin.
LDA (2 thành phần)	70 – 72%	< 1%	Phân biệt lớp tốt nhất, ổn định, tổng quát hóa cao.

7.3.2 Đánh giá theo nhóm dữ liệu

Kết luận: Mô hình Softmax hoạt động tốt nhất khi dữ liệu đã được chuẩn hóa và giảm chiều bằng **LDA**, do đặc trưng của LDA phù hợp với bản chất tuyến tính của Softmax Regression.

7.3.3 Đánh giá theo tỉ lệ train:test

Các tỷ lệ 4 : 1 (80 : 20), 7 : 3 (70 : 30) và 6 : 4 (60 : 40) đều được thử nghiệm.

- Tỷ lệ 80 : 20 cho kết quả **cao nhất và ổn định nhất**.
- Tỷ lệ 70 : 30 cho kết quả gần tương đương, chứng tỏ mô hình có khả năng tổng quát hóa tốt.
- Tỷ lệ 60 : 40 giảm nhẹ do lượng dữ liệu huấn luyện ít hơn, làm tăng sai số dự đoán.

Nhìn chung, mô hình **không bị overfitting**, vì chênh lệch độ chính xác giữa train và test đều nhỏ (< 3%).

=> Cấu hình tốt nhất

Bảng 9: Cấu hình tối ưu cho mô hình Softmax Regression

Thông số	Giá trị
Phương pháp	Softmax Regression (multinomial)
Dữ liệu	LDA
Tỷ lệ train:test	80:20
Solver	lbfgs
Accuracy	71.56%
Sai lệch	0.45%

7.3.4 Phân tích ma trận nhầm lẫn và phân bố dự đoán

- Lớp **Standard** được dự đoán chính xác cao nhất, chiếm tỷ lệ đúng khoảng 75 – 80%, do có nhiều mẫu huấn luyện hơn.
- Lớp **Poor** và **Good** thường bị nhầm thành **Standard**, cho thấy ranh giới giữa các lớp chưa thực sự tách biệt rõ.

Điều này phản ánh rằng mô hình **tuyến tính** như Softmax chỉ mô hình hóa được ranh giới đơn giản, nên khó xử lý các mối quan hệ phi tuyến phức tạp giữa các đặc trưng.

7.3.5 Nhận xét tổng quát

- Softmax Regression cho kết quả **ổn định**, diễn giải rõ ràng, đặc biệt hiệu quả khi dữ liệu được chuẩn hóa và giảm chiều bằng LDA.
- Mức chênh lệch train–test thấp chứng tỏ mô hình **không bị overfitting**.
- Tuy nhiên, độ chính xác tổng thể vẫn chỉ đạt khoảng 70%, cho thấy đặc trưng dữ liệu tín dụng có thể chứa quan hệ phi tuyến mà mô hình tuyến tính chưa mô tả hết được.

7.3.6 Kết luận

Softmax Logistic Regression là một phương pháp phân loại tuyến tính mạnh mẽ, dễ triển khai và có khả năng diễn giải cao. Trong bài toán dự đoán điểm tín dụng, mô hình này cho kết quả khá tốt, đặc biệt khi kết hợp với giảm chiều LDA, giúp cải thiện độ chính xác lên đến $\sim 70\%$ mà không xảy ra overfitting đáng kể. Tuy chưa đạt mức độ chính xác tuyệt đối, nhưng Softmax Regression vẫn là mô hình cơ sở (*baseline*) rất hữu ích để so sánh với các phương pháp phức tạp hơn trong học máy.

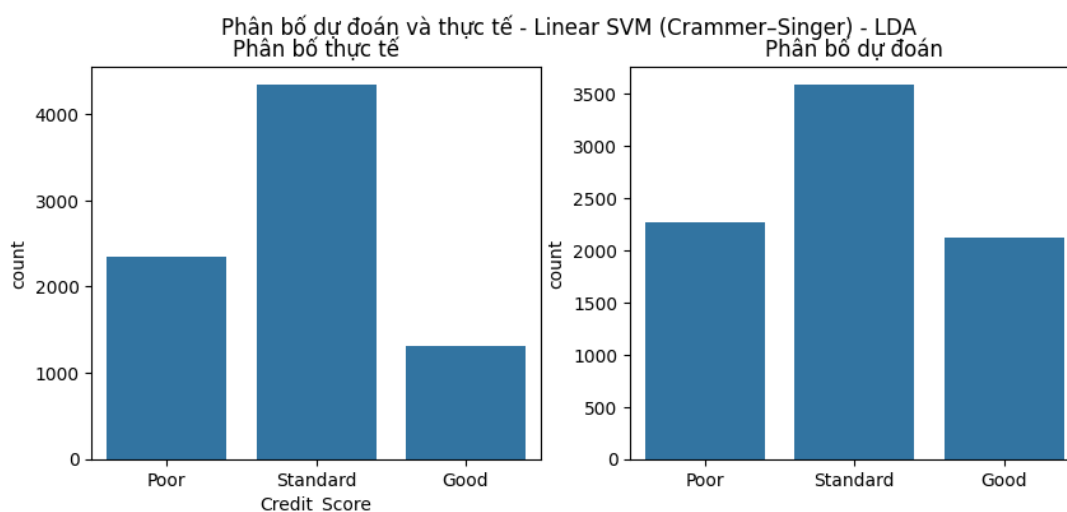
8 Đánh giá mô hình Crammer-Singer SVM

8.1 Kết quả thực nghiệm

8.1.1 Tỷ lệ train:test = 80:20

Bảng 10: Kết quả đánh giá mô hình với tỷ lệ 80:20

Loại dữ liệu	Accuracy Train	Accuracy Test	Chênh lệch
Gốc	69.54%	69.11%	0.43%
PCA	63.34%	63.18%	0.16%
LDA	71.37%	70.86%	0.51%

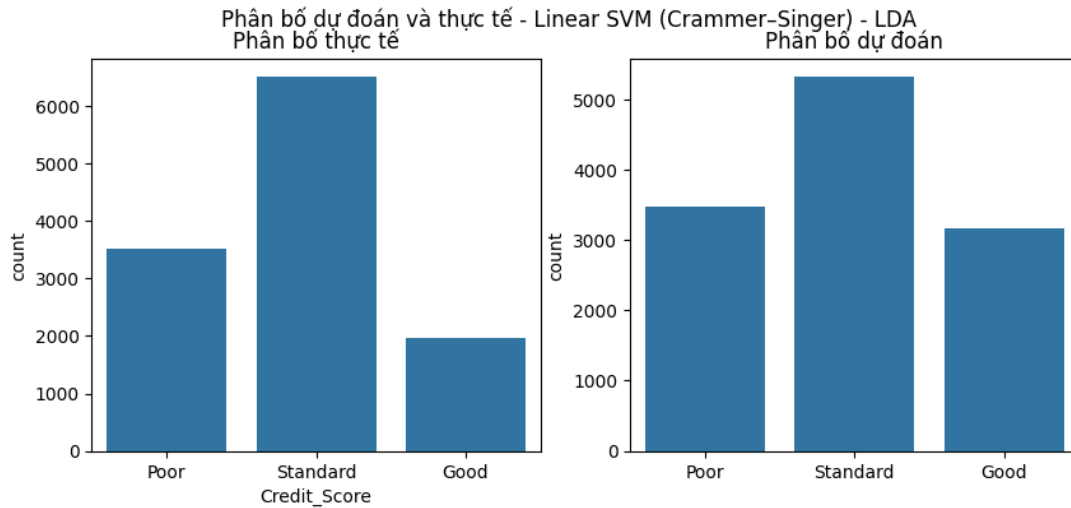


Hình 28: Kết quả tỷ lệ 8:2

8.1.2 Tỷ lệ train:test = 70:30

Bảng 11: Kết quả đánh giá mô hình với tỷ lệ 70:30

Loại dữ liệu	Accuracy Train	Accuracy Test	Chênh lệch
Gốc	69.58%	69.16%	0.42%
PCA	63.46%	63.12%	0.34%
LDA	71.45%	70.86%	0.59%

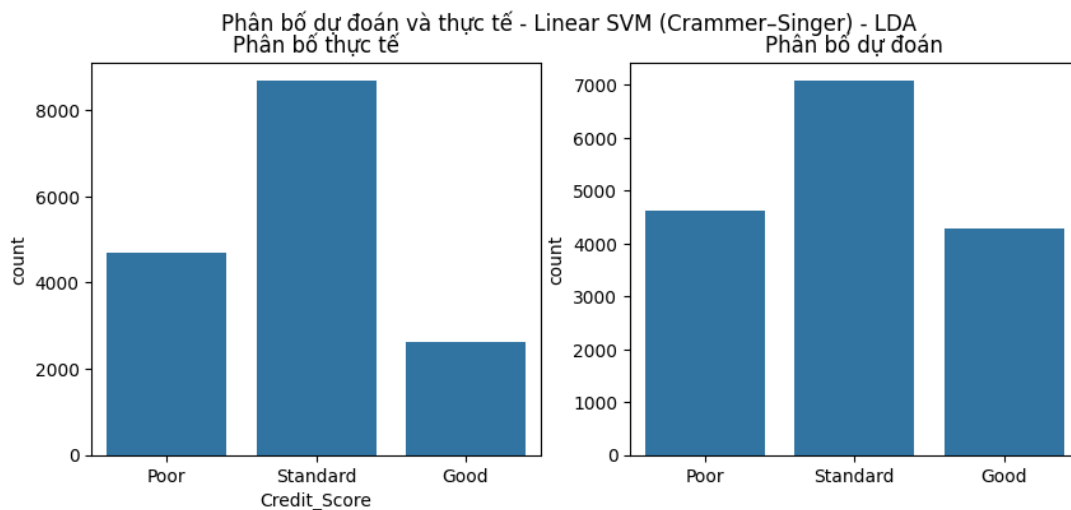


Hình 29: Kết quả tỉ lệ 7:3

8.1.3 Tỷ lệ train:test = 60:40

Bảng 12: Kết quả đánh giá mô hình với tỷ lệ 60:40

Loại dữ liệu	Accuracy Train	Accuracy Test	Chênh lệch
Gốc	63.84%	63.24%	0.6%
PCA	62.70%	62.39%	0.31%
LDA	71.69%	70.89%	0.8%



Hình 30: Kết quả tỉ lệ 6:4

Phân tích kết quả:

- Dựa trên các kết quả thu được, mô hình Linear SVM (Crammer-Singer, $C = 1.0$) đạt độ chính xác khoảng 69% trên tập kiểm tra, cho thấy hiệu suất ổn định và nhất quán giữa tập huấn luyện và kiểm tra, trong đó biến đổi LDA giúp duy trì mức accuracy cao nhất, còn PCA làm suy giảm đáng kể độ chính xác của mô hình.
- Output tương đối phù hợp với phân bố nhãn thực tế, tuy nhiên vẫn tồn tại sự sai lệch nhẹ về tỷ trọng giữa các lớp, đặc biệt ở các lớp Poor và Good, phản ánh hạn chế nhất định của mô hình trong việc tái hiện chính xác cấu trúc phân bố ban đầu của dữ liệu.

Nhận xét tổng quan

- Độ ổn định: Chênh lệch nhỏ giữa accuracy train và test chứng tỏ mô hình ổn định và ít bị overfitting
- Phân bố dự đoán: Phân bố nhãn dự đoán nhìn chung bám sát phân bố thực tế nhưng vẫn còn sai lệch nhẹ giữa các lớp.

Kết luận: Mô hình cho kết quả ở mức khá 70%, độ ổn định cao mà không bị overfitting. Linear SVM phù hợp làm mô hình nền (baseline) cho bài toán phân loại nhiều lớp có kích thước dữ liệu lớn, tuy nhiên hiệu suất có thể được cải thiện thêm bằng kernel phi tuyến hoặc tinh chỉnh tham số.

9 So sánh các mô hình

Bảng 13: So sánh hiệu suất các mô hình

Mô hình	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.726	0.690	0.750	0.740
KNN	0.706	0.690	0.732	0.680
SVM	0.717	0.703	0.72	0.674

Đánh giá khả năng huấn luyện:

Sau khi thực hiện phân loại trên tập dữ liệu kiểm thử, hai mô hình **K-Nearest Neighbors (KNN)**, **Softmax Regression** và Support Vector Machine Kết quả cho thấy cả 3 mô hình đều đạt độ chính xác (accuracy) trong khoảng 0.65–0.73, thể hiện khả năng nhận dạng ở mức trung bình khá.

Cụ thể, mô hình **SVM** cho kết quả nhỉnh hơn đôi chút so với **KNN** và **Softmax Regression**, phản ánh khả năng khai thác hiệu quả các đặc trưng đầu vào và hạn chế sự phụ thuộc vào thước đo khoảng cách như trong KNN.

Mô hình KNN mặc dù có ưu điểm đơn giản và dễ hiểu, nhưng lại nhạy cảm với phân bố dữ liệu và có thể bị ảnh hưởng bởi nhiễu hoặc các đặc trưng không chuẩn hoá. Trong khi đó, Softmax Regression học được trọng số cho từng đặc trưng, nên thể hiện khả năng tổng quát hóa tốt hơn trong bài toán này.

Nhìn chung, kết quả trên phản ánh rằng dữ liệu có mức độ phân tách vừa phải giữa các lớp — chưa đủ rõ ràng để mô hình đơn giản như KNN đạt hiệu quả cao, nhưng cũng đủ để mô hình tuyến tính như Softmax khai thác được cấu trúc chung.

Đánh giá tốc độ huấn luyện:

Bên cạnh độ chính xác, tốc độ huấn luyện và thời gian suy luận cũng được xem xét nhằm đánh giá toàn diện hiệu năng của các mô hình. Kết quả cho thấy **Softmax Regression** và **SVM** thể hiện ưu thế rõ rệt về mặt hiệu năng tính toán..

Cụ thể, **Softmax Regression** và **SVM** có thời gian huấn luyện và dự đoán nhanh hơn đáng kể so với **KNN**, do đây là mô hình tuyến tính chỉ cần học một tập trọng số cố định trong quá trình huấn luyện. Ngược lại, **KNN** không có pha huấn luyện rõ ràng mà phải lưu trữ toàn bộ dữ liệu huấn luyện và thực hiện tính toán khoảng cách với tất cả các điểm dữ liệu mỗi khi dự đoán, dẫn đến chi phí tính toán cao hơn đáng kể khi kích thước tập dữ liệu tăng.

10 Mở rộng sang bài toán hồi quy

10.1 Mô hình SVR

10.1.1 Ý tưởng

Bài toán phân loại ban đầu được chuyển đổi sang bài toán hồi quy thông qua việc thay đổi hàm mục tiêu của mô hình SVM. Cụ thể, thay vì tối ưu hóa hàm mất mát cho phân loại, mô hình được xây dựng dưới dạng Support Vector Regression (SVR), trong đó mục tiêu là tìm một hàm hồi quy sao cho sai lệch giữa giá trị dự đoán và giá trị thực của biến mục tiêu được tối thiểu hóa trong một ngưỡng cho phép.

10.1.2 Hàm mục tiêu

Cụ thể, việc biến đổi SVM từ bài toán phân loại sang bài toán hồi quy được thực hiện thông qua thay đổi hàm mục tiêu và ràng buộc tối ưu của mô hình

SVR xây dựng một hàm hồi quy

$$f(x) = \mathbf{w}^T \mathbf{x} + b$$

sao cho sai lệch giữa giá trị dự đoán và giá trị thực không vượt quá một ngưỡng ε đối với phần lớn các mẫu dữ liệu. Hàm mục tiêu của SVR được thiết kế nhằm tối thiểu hóa độ phức tạp của mô hình (thông qua chuẩn $\|\mathbf{w}\|^2$), đồng thời phạt các sai lệch vượt quá ngưỡng ε .

Bài toán tối ưu của SVR có dạng:

$$\min_{\mathbf{w}, b, \xi, \xi^*} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N (\xi_i + \xi_i^*)$$

Với các ràng buộc:

$$\begin{cases} y_i - (\mathbf{w}^T \mathbf{x}_i + b) \leq \varepsilon + \xi_i, \\ (\mathbf{w}^T \mathbf{x}_i + b) - y_i \leq \varepsilon + \xi_i^*, \\ \xi_i, \xi_i^* \geq 0, \quad i = 1, \dots, N. \end{cases}$$

Trong đó, ε xác định **độ rộng của vùng không phạt lỗi** (ε -insensitive tube), còn tham số C điều chỉnh mức độ đánh đổi giữa độ chính xác hồi quy và khả năng tổng quát hóa của mô hình.

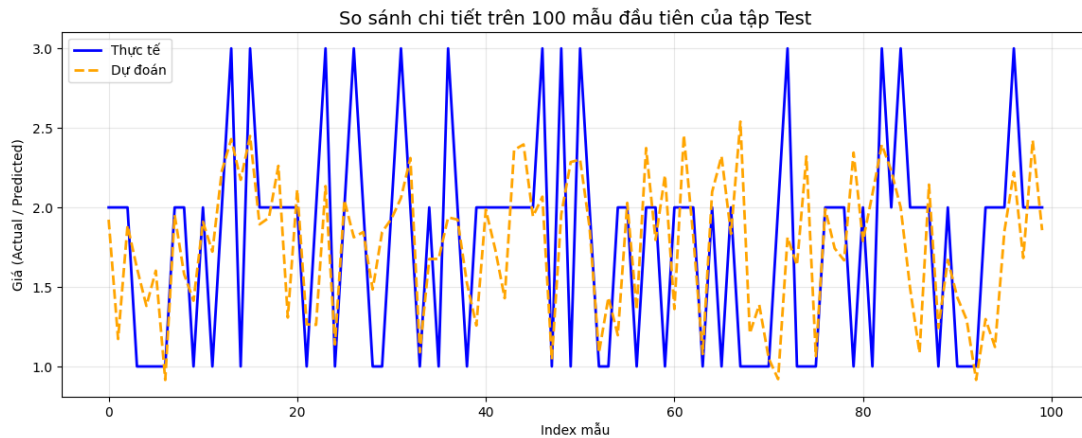
Như vậy, việc chuyển đổi từ SVM sang SVR tương đương với việc thay thế hàm mất mát phân loại bằng hàm mất mát ε -insensitive, từ đó cho phép mô hình học được mối quan hệ liên tục giữa các đặc trưng đầu vào và biến mục tiêu đã được mã hóa theo thứ tự.

10.1.3 Kết quả thực nghiệm

Tỉ lệ train:test = 8:2

Bảng 14: Kết quả đánh giá mô hình với tỷ lệ 80:20

Loại dữ liệu	MSE	RMSE	R Square
Gốc	0.41	0.63	0.091
PCA	0.32	0.56	0.26
LDA	0.29	0.54	0.33

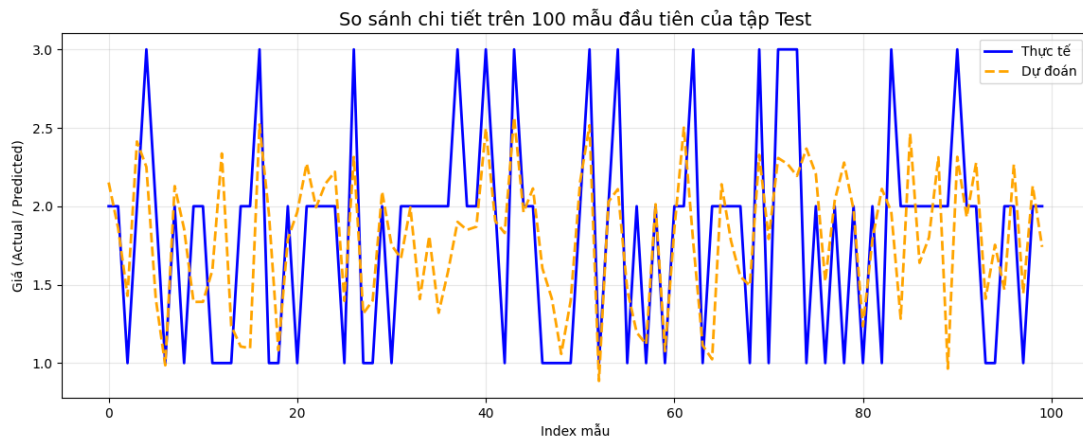


Hình 31: Kết quả tỉ lệ 6:4

Tỉ lệ train:test = 7:3

Bảng 15: Kết quả đánh giá mô hình với tỷ lệ 70:30

Loại dữ liệu	MSE	RMSE	R Square
Gốc	0.395	0.63	0.097
PCA	0.32	0.56	0.27
LDA	0.29	0.55	0.34

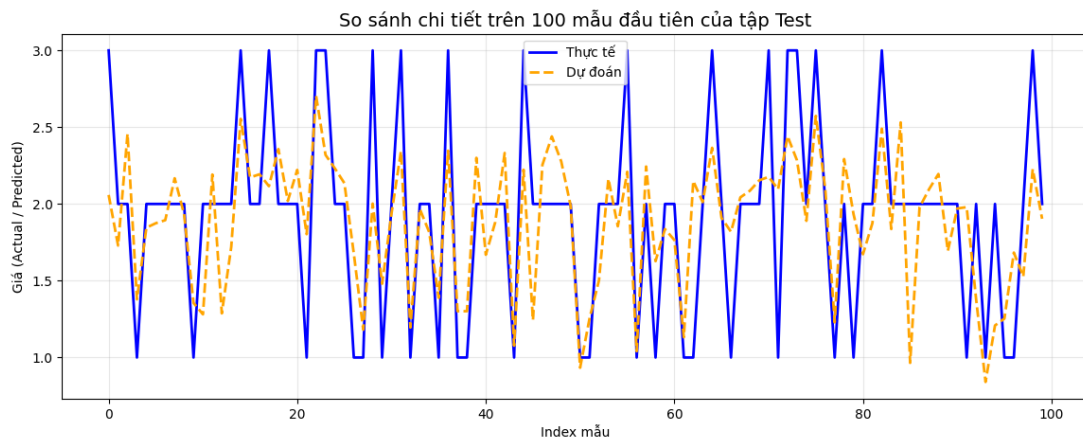


Hình 32: Kết quả tỉ lệ 6:4

Tỉ lệ train:test = 6:4

Bảng 16: Kết quả đánh giá mô hình với tỷ lệ 60:40

Loại dữ liệu	MSE	RMSE	R Square
Gốc	0.4	0.63	0.088
PCA	0.32	0.56	0.27
LDA	0.29	0.54	0.34



Hình 33: Kết quả tỉ lệ 6:4

10.1.4 Phân tích kết quả

- Mô hình cho kết quả còn hạn chế với sai số tương đối lớn và giá trị R^2 thấp, cho thấy khả năng giải thích phương sai của dữ liệu chưa cao.
- Việc giảm chiều bằng PCA giúp cải thiện sai số dự đoán và nâng cao hiệu suất mô hình so với dữ liệu gốc, tuy nhiên mức cải thiện vẫn còn ở mức trung bình.
- LDA cho kết quả tốt nhất với MSE và RMSE thấp nhất, R^2 cao nhất.

10.1.5 Đánh giá tổng quan

- Hiệu suất dự đoán thấp: Mô hình SVR (LinearSVR) cho giá trị R^2 thấp (dưới 0.35 ngay cả với LDA), cho thấy khả năng giải thích phương sai của biến mục tiêu còn rất hạn chế.
- Sai số vẫn đáng kể: Mặc dù MSE và RMSE có cải thiện khi kết hợp với PCA và LDA, mức sai số dự đoán vẫn còn cao, phản ánh độ chính xác hồi quy chưa đáp ứng yêu cầu thực tiễn.
- SVR không mang lại lợi ích rõ rệt về mặt hiệu suất, dù có ý nghĩa về mặt phương pháp.

Kết luận: Trong bối cảnh bài toán này, SVR hoạt động kém và chỉ mang tính thử nghiệm, chưa phù hợp để sử dụng như mô hình dự đoán chính.

Trong phần này, bài toán phân loại điểm tín dụng được mở rộng sang bài toán hồi quy bằng cách khai thác giá trị hàm quyết định của mô hình phân loại Softmax Regression. Thay vì chỉ sử dụng nhãn phân loại rời rạc, cách tiếp cận này cho phép mô hình hóa mức độ tin cậy của dự đoán dưới dạng giá trị liên tục, từ đó cung cấp thêm thông tin phục vụ phân tích và ra quyết định.

10.2 Mô hình hồi quy dựa trên Softmax

Trong phần này, bài toán phân loại điểm tín dụng được mở rộng sang bài toán hồi quy bằng cách khai thác giá trị hàm quyết định của mô hình phân loại Softmax Regression. Thay vì chỉ sử dụng nhãn phân loại rời rạc, cách tiếp cận này cho phép mô hình hóa mức độ tin cậy của dự đoán dưới dạng giá trị liên tục, từ đó cung cấp thêm thông tin phục vụ phân tích và ra quyết định.

10.2.1 Ý tưởng

Trong bài toán phân loại đa lớp bằng Softmax Regression, đầu ra của mô hình là phân phối xác suất trên các lớp:

$$P(y = k | x), \quad k \in \{\text{Poor}, \text{Standard}, \text{Good}\}.$$

Thay vì chỉ lựa chọn lớp có xác suất lớn nhất, ta chọn một phân lớp cụ thể (trong bài này là lớp **Good**) và sử dụng xác suất Softmax tương ứng làm đầu ra hồi quy:

$$y_{\text{reg}} = P(y = \text{Good} | x).$$

Giá trị $y_{\text{reg}} \in [0, 1]$ phản ánh mức độ tin cậy của mô hình rằng một khách hàng thuộc nhóm tín dụng tốt. Như vậy, bài toán được chuyển từ phân loại sang bài toán hồi quy liên tục.

10.2.2 Hàm mục tiêu

Với đầu ra mới y_{reg} , mục tiêu của bài toán hồi quy là học hàm:

$$f(x) \approx P(y = \text{Good} | x).$$

Hai mô hình hồi quy được sử dụng trong thực nghiệm gồm Ridge Regression và Support Vector Regression (SVR) với kernel RBF.

Ridge Regression Ridge Regression là mô hình hồi quy tuyến tính có hiệu chỉnh chuẩn L_2 , với hàm mất mát:

$$J(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (y_i - \mathbf{w}^T x_i)^2 + \lambda \|\mathbf{w}\|_2^2.$$

Trong đó, thành phần chuẩn hóa L_2 giúp giảm hiện tượng overfitting và đặc biệt phù hợp với dữ liệu có số chiều lớn hoặc tồn tại đa cộng tuyến.

Support Vector Regression (SVR) với kernel RBF SVR cho phép mô hình hóa các quan hệ phi tuyến thông qua hàm kernel. Trong bài này, kernel RBF được sử dụng:

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2).$$

SVR tìm hàm hồi quy sao cho sai số nằm trong một ống ε là nhỏ nhất, đồng thời tối ưu độ phức tạp của mô hình.

10.2.3 Kết quả thực nghiệm

Thực nghiệm được tiến hành với các tỉ lệ chia dữ liệu giống phần phân loại:

- Train : Test = 8 : 2
- Train : Test = 7 : 3
- Train : Test = 6 : 4

Mỗi mô hình hồi quy được đánh giá trong hai trường hợp:

- Dữ liệu đầu vào nguyên bản (đã chuẩn hóa)
- Dữ liệu giảm chiều còn 1/3 số đặc trưng bằng PCA

Các chỉ số đánh giá được sử dụng bao gồm:

- RMSE (Root Mean Squared Error)
- MAE (Mean Absolute Error)
- R^2 -score

Kết quả thực nghiệm cho thấy Ridge Regression cho kết quả ổn định giữa các tỉ lệ chia dữ liệu, trong khi SVR (RBF) đạt sai số thấp hơn trong một số trường hợp, đặc biệt khi dữ liệu được giảm chiều.

	split	input	model	rmse	mae	r2
0	8:2	Original_std	Ridge	0.057561	0.042143	0.930969
1	8:2	PCA_1over3(d=16)	Ridge	0.105374	0.077793	0.768658
2	8:2	Original_std	SVR_RBF	0.051860	0.043494	0.943965
3	8:2	PCA_1over3(d=16)	SVR_RBF	0.081112	0.060066	0.862926
4	7:3	Original_std	Ridge	0.059540	0.043667	0.927380
5	7:3	PCA_1over3(d=16)	Ridge	0.107430	0.079302	0.763573
6	7:3	Original_std	SVR_RBF	0.052655	0.044369	0.943202
7	7:3	PCA_1over3(d=16)	SVR_RBF	0.082332	0.059998	0.861138
8	6:4	Original_std	Ridge	0.059006	0.043232	0.928500
9	6:4	PCA_1over3(d=16)	Ridge	0.106449	0.078806	0.767302
10	6:4	Original_std	SVR_RBF	0.053703	0.045277	0.940775
11	6:4	PCA_1over3(d=16)	SVR_RBF	0.082473	0.060268	0.860320

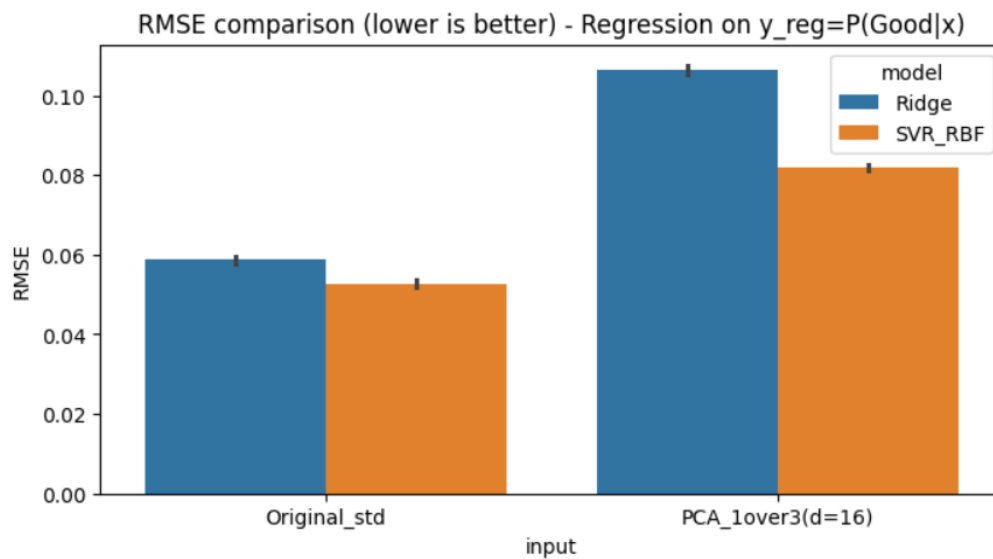
Hình 34: Kết quả đánh giá mô hình với các tỷ lệ

10.2.4 Phân tích kết quả

Đối với Ridge Regression, do bản chất tuyến tính nên mô hình khó nắm bắt các quan hệ phi tuyến phức tạp giữa các đặc trưng tài chính và xác suất Softmax. Tuy nhiên, Ridge Regression có ưu điểm là ổn định, ít nhạy cảm với thay đổi tỉ lệ train:test và ít bị overfitting.

Đối với SVR (RBF), khả năng mô hình hóa phi tuyến giúp mô hình bám sát phân bố của y_{reg} . Tuy nhiên, SVR nhạy cảm với việc lựa chọn tham số và phân bố dữ liệu. Khi dữ liệu được giảm chiều bằng PCA, hiệu quả của SVR thường được cải thiện do giảm nhiễu và tăng khả năng tổng quát hóa.

Việc giảm chiều về 1/3 số đặc trưng giúp cân bằng giữa việc giữ lại thông tin quan trọng và hạn chế overfitting, đặc biệt có lợi cho các mô hình phi tuyến.



Hình 35: So sánh kết quả

10.2.5 Đánh giá tổng quan

Việc mở rộng bài toán phân loại sang hồi quy dựa trên hàm quyết định Softmax mang lại nhiều lợi ích:

- Khai thác thông tin xác suất thay vì chỉ nhãn rời rạc
- Đánh giá mức độ tín nhiệm của khách hàng theo thang liên tục
- Phù hợp với các bài toán xếp hạng tín dụng và ra quyết định mềm

So sánh hai mô hình hồi quy cho thấy:

- Ridge Regression: đơn giản, ổn định và dễ diễn giải
- SVR (RBF): mạnh hơn trong trường hợp dữ liệu có quan hệ phi tuyến

Tổng kết, phương pháp hồi quy dựa trên Softmax là hợp lý, có cơ sở toán học rõ ràng và bổ sung góc nhìn quan trọng cho bài toán đánh giá điểm tín dụng.

Tài liệu tham khảo

- [1] Trang web Machine learning cơ bản: <https://machinelearningcoban.com>
- [2] Kênh youtube Statquest: <https://www.youtube.com/@statquest>
- [3] Sách "**Machine learning**" - tác giả **Tom Mitchell**